

Object-Oriented Programming

5.1	Software Reuse and Independence	143
5.2	Smalltalk	144
5.3	Java	162
5.4	C++	181
5.5	Design Issues in Object-Oriented Languages	191
5.6	Implementation Issues in Object-Oriented Languages	195

CHAPTER 5

The history of object-oriented programming languages begins in the 1960s with the Simula project, an attempt to design a programming language that extended Algol60 in a way suitable for performing computer simulations of real-world situations. One of the central goals of this project was to incorporate into the language the notion of an object, which, similar to a real-world object, is an entity with certain properties that control its ability to react to events in predefined ways. According to this way of looking at programming, a program consists of a set of objects, which can vary dynamically, and which execute by acting and reacting to each other, in much the same way that a real-world process proceeds by the interaction of real-world objects. These ideas were incorporated into the general-purpose language Simula67, at the Norwegian Computing Center in Oslo, by Ole-Johan Dahl and Kristen Nygaard.

By the 1970s, the programming world was ready to incorporate some elements of Simula67 into new languages. Its influence was twofold. It was, first of all, an important factor in the development of abstract data type mechanisms, which you will study in Chapter 11. Second, and perhaps more importantly, it was crucial to the development of the object paradigm itself, in which a program is considered to be a collection of interacting independent objects. This is most evident in the development of the Dynabook Project, which culminated in the language Smalltalk-80, the first language to incorporate the object paradigm in a thorough and consistent way.

Beginning in the mid-1980s, interest in **object-oriented programming** exploded, just like interest in structured programming and top-down design in the early 1970s. Language designers focused on both object-oriented programming's utility as a language paradigm and its usefulness as a methodology for program design. These days, almost every language has some form of structured constructs, such as the `if-then-else` statement and procedural abstraction, and it is well known that it is possible to apply structured programming principles even in a relatively unstructured language like FORTRAN. Similarly, the ideas of object-oriented programming can be applied, at least to a certain extent, in non-object-oriented languages. Nevertheless, the real power of the technique comes only in a language with true object-oriented constructs, and over the last three decades object-oriented programming has proven itself to be an extremely effective mechanism for promoting code reuse and modifiability. It is now the dominant paradigm for large software projects, despite certain drawbacks in a few areas, such as its ability to express abstract data types.

In the following sections, we will review the major concepts of object-oriented programming, using Smalltalk as our primary example. The concepts we will cover include the notions of object and class as a pattern for objects, inheritance of operations as a tool for code reuse and the maintenance of control over dependencies, and the dynamic nature of operations as an essential feature of reuse. We will also study two other languages as major examples of the object-oriented paradigm: Java and C++. Finally, we will look briefly at some issues of object-oriented design and techniques for the implementation of object-oriented language features.

5.1 Software Reuse and Independence

Object-oriented programming languages satisfy three important needs in software design: the need to reuse software components as much as possible, the need to modify program behavior with minimal changes to existing code, and the need to maintain the independence of different components. Abstract data type mechanisms can increase the independence of software components by separating interfaces from implementations and by controlling dependencies. In theory, abstract data type mechanisms should also provide for the reuse of software components. In practice, however, each new programming problem tends to require a slightly different version of the services provided by a module. Thus, the ideal solution varies the services a module offers to clients, while at the same time retaining control over access to those services. In this section, we explore the principal ways that the services of software components can be varied. These can then be used to evaluate the versatility and effectiveness of an object-oriented mechanism in the subsequent sections.

There are four basic ways that a software component can be modified for reuse: extension of the data or operations, redefinition of one or more of the operations, abstraction, and polymorphism:

- *Extension of the data or operations.* As an example of this kind of modification, consider a queue with the following operations: `create`, `enqueue`, `dequeue`, `front`, and `empty`. In a particular application, it may be necessary to extend the operations of a queue so that elements can be removed from the rear of the queue and added to the front of the queue. Such a data structure is called a double-ended queue or deque, with new operations `addfront` and `deleterear`. These two new operations need to be added to the basic queue structure without necessarily changing the underlying implementation data. As another example of modification by extension, a window is defined on a computer screen as a rectangle specified by its four corners, with operations that may include `translate`, `resize`, `display`, and `erase`. A text window can be defined as a window with some added text to be displayed. Thus, a text window extends a window by adding data, without necessarily changing the operations to be performed.
- *Redefinition of one or more of the operations.* Even if the operations on a new type of data remain essentially the same as those on existing types, it may be necessary to redefine some of them to accommodate new behavior. For example, a text window may have the same operations on it as a general-purpose window, but the `display` operation needs redefinition in order to display the text in the window as well as the window itself. Similarly, if a square is obtained from a rectangle, an area or perimeter function may need to be redefined to take into account the reduced data needed in the computation.
- In several areas in computing, the basic structure of each application is so similar to others that software developers have begun using **application frameworks**. An application framework is a collection of related software resources, usually in object-oriented form, that is used

by software developers through redefinition and reuse to provide specific services for applications. Examples of application frameworks include the **Swing** windowing toolkit in Java and **Microsoft Foundation Classes** in C++. Both of these frameworks allow graphical user interfaces to be constructed, modified, and reused with a minimum of effort.

- *Abstraction, or the collection of similar operations from two different components into a new component.* For example, a circle and a rectangle are both objects that have position and that can be translated and displayed. These properties can be combined into an abstract object called a figure, which has the common features of circles, rectangles, triangles, and so on. Specific examples of a figure are then forced to have each of these common properties.
- *Polymorphism, or the extension of the type of data that operations can apply to.* You have already seen examples of this in previous chapters as two kinds of polymorphism: overloading and parameterized types. Extending the types that an operation applies to can also be viewed as an example of abstraction, where common operations from different types are abstracted and gathered together. A good example is a `toString` function, which should be applicable to any object as long as it has a textual representation.

Design for reuse is not the only goal of object-oriented languages. Restricting access to internal details of software components is another. This requirement is necessary to ensure that clients use components in a way that is independent of the implementation details of the component and that any changes to the implementation of a component will have only a local effect.

The two goals of restricting access and allowing modifiability for reuse can be mutually incompatible. For example, to add operations to a queue, one needs access to the internal implementation details of the queue's data. On the other hand, access restriction can also be complementary to the need for modification. Access restriction can force operations to be defined in an abstract way, which may make it easier to modify their implementation without changing the definition.

Mechanisms for restricting access to internal details go by several names. Some authors call them **encapsulation mechanisms**, while others refer to them as **information-hiding mechanisms**. We now examine how these concepts are realized in a pure object-oriented language, Smalltalk.

5.2 Smalltalk

Smalltalk arose out of a research project, called the Dynabook Project, begun by Alan Kay at Xerox Corporation's Palo Alto Research Center in the early 1970s and continued by Adele Goldberg and Daniel Ingalls. The Dynabook, in theory at least, was conceived as the prototype of today's laptop and tablet computers. The system would include a WIMP (windows, icons, mouse, pull-down menus) user interface and be networked to other computers for group activity. Ideally, it would include a program development environment capable of modeling all of the interactions among these entities. The vision of this research

group was ahead of its time; its realization had to await the turn of the present century, when hardware costs and performance made it a realistic basis for contemporary computing.

Smalltalk was influenced by both Simula and Lisp. A series of stages marked its initial development, with its principal early versions being Smalltalk-72, Smalltalk-76, and Smalltalk-80. By the late 1980s, ordinary desktop computers had become powerful enough to support the memory-intensive Smalltalk programming environment. A company named Digitalk released Smalltalk/V, which ran on both Windows and Macintosh platforms. Smalltalk achieved an ANSI standard in 1998, and several open source and low-cost commercial versions are currently available.

Of all the object-oriented languages, Smalltalk has the most consistent approach to the object-oriented paradigm. In Smalltalk, almost every language entity is an object, including constants (such as numbers and characters) and classes themselves. Thus, Smalltalk can be said to be **purely** object oriented in essentially the same sense that languages such as Haskell can be called purely functional.

Smalltalk borrowed from Lisp the mechanisms of garbage collection (automatic recycling of unused dynamic storage at runtime) and dynamic typing (the checking of the types of operands in expressions at runtime). However, Smalltalk was innovative not only from the language point of view but also in that, from the beginning, it was envisioned as a complete program development environment for a personal workstation. The user interface was also novel in that it included a windowing system with menus and a mouse, long before such systems became common for personal computers.

Smalltalk as a language is interactive and dynamically oriented. Programmers can experiment by entering expressions and statements in an interactive interpreter. All classes and objects are created by interaction with the system, using a set of browser windows. New classes are compiled as they are entered into a class dictionary, and the interaction of objects is also managed by the system. The Smalltalk system comes with a large hierarchy of preexisting classes, and each new class must be a subclass of an existing class. In the following subsections, we explore the syntax of Smalltalk expressions and statements, the definition of new classes, and the fundamental object-oriented concepts of message passing, encapsulation, inheritance, and polymorphism.

5.2.1 Basic Elements of Smalltalk: Classes, Objects, Messages, and Control

Every object in the real world has a set of properties and behaviors. For example, a car has properties that include the number of wheels, the number of doors, and mileage rating. Its behaviors include moving forward, moving in reverse, and turning left or right. Likewise, every object in Smalltalk (that is, every data value and every class or data type), whether it is built-in or created by the programmer, has a set of properties and behaviors. In a Smalltalk program, objects get other objects to do things by sending them messages. A **message** is a request for a service. The object that receives a message (sometimes called a **receiver**) performs its service by invoking an appropriate **method**. This method may in turn send messages to other objects to accomplish its task. As you will see shortly, a method is like a procedure or function, and is defined in the receiver object's class or one of its ancestor classes in a class hierarchy. The **sender** of a message may also supply data with it, in the form of parameters or arguments. When the receiver's method call is completed, it may return data (another object) to the sender. Thus, sending

a message to an object is a bit like calling a function in a functional language. However, some messages, called **mutators**, may result in a change of state in the receiver object, whereas a function call simply returns a value to the caller. The process of sending and receiving messages is also called **message passing**. The set of messages that an object recognizes is sometimes called its **interface**.

The syntax of Smalltalk message passing is quite simple but somewhat unconventional. The object receiving the message is written first, followed by the message name (also called a **selector**) and any arguments. The elements of an expression containing objects and messages are typically read and evaluated from left to right. For example, we can create a new set object by sending the message `new` to Smalltalk's built-in `Set` class, as follows:

```
Set new      "Returns a new set object"
```

Note that a class name is capitalized and a message name is not. Smalltalk comments are enclosed in double quotation marks. Any of the code examples in this subsection can be evaluated by entering them in a Smalltalk transcript window and selecting the **Show it** option from the **Smalltalk** menu. We use the Smalltalk/V syntax throughout this discussion.

We can examine the number of elements in a new set by sending the `size` message to this object:

```
Set new size      "Returns 0"
```

In this code, the `Set` class receives the `new` message, which returns an instance of `Set`, as before. This object in turn receives the `size` message, which returns an integer object. The message `new` is also called a **class message**, because it is sent to a class, whereas the message `size` is called an **instance message**, because it is sent to an instance of a class. These messages are also examples of **unary messages**, in that they expect no arguments.

Messages that expect arguments are called **keyword messages**. For example, we can see whether a new set contains a given element by sending it the `includes:` message. This message expects a single argument, the object that is the target of the search. In the next example, a set is asked if it includes the string 'Hello':

```
Set new includes: 'Hello'      "Returns false"
```

Note that the name of a keyword message ends in a colon (:). If there is more than one argument, another keyword must precede each argument. The arguments to a keyword message are evaluated before the message is sent. The following example adds a string to a new set object:

```
Set new add: 'Ken'      "This set now includes 'Ken'"
```

The `add:` message is an example of a **mutator**, which changes the state of the receiver object.

A slightly more complicated example creates a new array of 10 elements and stores the value 66 at the first position in the array. The code first sends the class message `new:` to the `Array` class to instantiate an array of 10 positions (each of which contains the default value `nil`). This array then receives the instance message `at:put:` to store an integer at the array's first position.

```
(Array new: 10) at: 1 put: 66      "Stores 66 at the first position in the array"
```

The keyword message `at:put:` expects two arguments. The first one is the index of an object in the array. The second one is the object to be placed at that index. Unary messages have a higher precedence than keyword messages, and parentheses can be used for clarity or to override precedence. Note that the

last example requires parentheses to distinguish the two keyword messages. Without the parentheses, the interpreter would treat `new:at:put` as a class message for the `Array` class. This would generate an unrecognized message error.

Smalltalk also provides a small set of **binary messages** that allow the programmer to write arithmetic and comparison expressions using conventional infix notation. For example, each of the expressions:

```
3 + 4          "Returns 7"
3 < 4          "Returns true"
```

is actually syntactic sugar for a keyword message that is sent to the first integer object (the receiver). The second integer object is the argument to the message. Binary messages have a lower precedence than keyword messages.

As in many other languages, the programmer can use variables to refer to objects. They can also use sequences of statements to organize code. The next code segment assigns a new array of five positions to a temporary variable and then sends five messages in separate statements to reset its contents:

```
| array |
array := Array new: 5.      "Create an array of 5 positions"
array at: 1 put: 10.        "Run a sequence of 5 replacements"
array at: 2 put: 20.
array at: 3 put: 30.
array at: 4 put: 40.
array at: 5 put: 50
```

Temporary variables are introduced between vertical bars and are not capitalized. Statements are separated by a period. As in Lisp, Smalltalk variables have no assigned data type; any variable can name anything. Note also that Smalltalk's assignment operator, `:=`, is written as in Pascal and Ada.

When a sequence of messages is sent to the same object, we can use a semicolon to cascade the messages:

```
| array |
array := Array new: 5.      "Create an array of 5 positions"
array at: 1 put: 10;        "Run a sequence of 5 replacements, cascaded"
    at: 2 put: 20;
    at: 3 put: 30;
    at: 4 put: 40;
    at: 5 put: 50
```

As in many other languages (Lisp, Java, and Python), Smalltalk's variables use **reference semantics** rather than **value semantics**. In reference semantics, a variable *refers* to an object, whereas in value semantics (see languages like Ada and C++), a variable *is* or *contains* an object. Thus, an assignment of one Smalltalk variable to another does not transfer a copy of the object itself, as in Ada or C++, but just establishes another reference (essentially an alias) to the same object. Copying requires an explicit instantiation of a new object with the same attributes as the original. Smalltalk includes two operators,

= for equality and == for object identity, to distinguish the two relations. They are illustrated in the next code segment, which uses Smalltalk's #() notation for a literal array:

```
| array alias clone |
array := #(0 0).      "Create an array containing two integers"
alias := array.       "Establish a second reference to the same object"
clone := #(0 0).      "Create a copy of the first array object"
array == alias.       "true, they refer to the exact same object"
array = alias.        "true, because they are =="
array = clone.        "true, because they refer to distinct objects with"
                     "the same properties"
array == clone        "false, because they refer to distinct objects"
clone at: 1 put: 1.   "The second array object now contains 1 and 0"
array = clone         "false, because the two array objects no longer"
                     "have the same properties"
```

The sharing of objects in reference semantics enhances the efficiency of object-oriented programs, at the cost of possible safety problems because of aliasing. This is not a problem for functional languages, because they allow no side effects on objects.

Ideally, the code to initialize array elements should run as a loop, which would work for an array of any size. The next code segment shows a to:do loop that sets the object at each position in an array to a new value. This loop is similar to an index-based for loop in other languages.

```
| array |
array := Array new: 100.      "Instantiate an array of 100 positions"
1 to: array size do: [:i |    "Index-based loop sets the element at each i"
    array at: i put: i * 10]
```

The message to:do: is sent to an integer, which represents the lower bound of the iteration. The argument associated with the keyword to: is also an integer, representing the iteration's upper bound (in this case, the size of the array). The argument associated with the keyword do: is a Smalltalk **block**. A block, which is enclosed in brackets, [], is an object that contains the code that executes on each pass through the loop. Blocks are similar to lambda forms in Scheme, in that they can contain arguments in the form of **block variables**. The to:do: block in our example contains the block variable i, which receives each value in the range between the lower bound and the upper bound during the execution of the loop. On each pass through the to:do: loop, Smalltalk sends the block the value: message, with the next number in the series as an argument. The remaining code in a block can be any sequence of statements. As you can see, even control statements are expressed in terms of message passing in Smalltalk.

Other types of control statements, such as if/else statements and while loops, are also expressed in terms of keyword messages in Smalltalk. The next code segment uses the ifTrue:ifFalse: message to set the array cells at the odd positions to 1 and at the even positions to 2:


```

| array |
array := Array new: 100.      "Instantiate an array of 100 positions"
1 to: array size do: [:i |    "Treat odd and even positions differently"
  i odd
    ifTrue: [array at: i put: 1]
    ifFalse: [array at: i put: 2]]

```

The receiver of the `ifTrue:ifFalse:` message must be a Boolean object. The two arguments are blocks of code that express the alternative courses of action in the `if/else` logic. The receiver object evaluates just one of the two blocks, based on its own state (either true or false). Note that the scopes of the temporary variable `array` and the block variable `i` in this example include any nested blocks.

Because the loop message `to:do:` is sent to a number, the code that implements its method is found in the `Number` class. We will discuss defining methods for classes shortly, but here is a preview, which shows that `to:do:` is really just syntactic sugar for a more cumbersome, count-controlled `whileTrue:` loop:

```

to: aNumber do: aBlock
  "Evaluate the one argument block aBlock for the
  numbers between the receiver and the argument
  aNumber where each number is the previous number
  plus 1."
  | index |
  index := self.
  [index <= aNumber]
    whileTrue: [
      aBlock value: index.
      index := index + 1]

```

In the context of this method definition, the symbol `self` refers to the receiver of the message, which in this case is a number. The `whileTrue:` message is associated with two blocks. Unlike the `ifTrue:ifFalse:` message, whose receiver object is evaluated just once, the `whileTrue:` message must have a block as its receiver object. Smalltalk evaluates this block, which contains a Boolean expression, one or more times, until it returns false. Each time the first block evaluates to true, the second block (`whileTrue:`'s argument), which represents the body of the loop, is also evaluated. The code in this second block consists of two statements. The first statement evaluates `aBlock` (the argument of the `to:do:` method being defined here) by sending it the `value:` message. During the course of the loop, the argument to this message (`index`) picks up the value of each number in the series between `self` and `aNumber`. The second statement increments `index` to achieve this effect.

The final example code segment in this subsection shows how to print the contents of an array to the transcript window. The code uses a `do:` loop to traverse the array. Here we are interested not in replacing elements at index positions within the array but in just visiting the elements themselves. The `do:` loop visits each of the array's elements in sequence, from the first to the last. On each pass through the loop, the block argument is evaluated with the block variable bound to the current element.

```
array do: [:element | "Print to transcript each element followed by a return"
  Transcript nextPutAll: element printString; cr]
```

The global variable `Transcript` refers to Smalltalk's transcript window object. The message `nextPutAll` expects a string as an argument, so we use the message `printString` to obtain the string representation of the element before sending it to the transcript with this message. The message `cr` asks the transcript to display a return character.

Smalltalk includes many types of collection classes and different types of messages for performing iterations on collections, such as maps and filters, similar to those of functional languages. We discuss some of these operations, also called **iterators**, shortly.

As you can see from these brief examples, most Smalltalk code consists of creating objects of the appropriate classes and sending them messages. The objects themselves handle most of the algorithmic details. The rest of software development then consists in choosing the right classes for the right jobs and learning their interfaces. When such classes do not already exist, the Smalltalk programmer creates new classes that customize existing classes. To see how this is done, we now explore two categories of classes, magnitudes and collections, in Smalltalk's massive built-in class hierarchy.

5.2.2 The Magnitude Hierarchy

A class provides a definition of the properties and behavior of a set of objects. In Smalltalk, the built-in classes are organized in a tree-like hierarchy, with a single class, called `Object`, at the root. Each class below `Object` inherits any properties and behavior defined in any classes that are its ancestors in the hierarchy. The classes in the hierarchy usually descend from the more general to the more specific, with each new class adding to and/or modifying the behavior and/or the properties of those already available in the classes above it. **Inheritance**, thus, supports the reuse of structure and behavior. **Polymorphism**, or the use of the same names for messages requesting similar services from different classes, is also a form of code reuse. In this section, we examine a portion of Smalltalk's magnitude classes, which are shown in the class diagram in Figure 5.1.

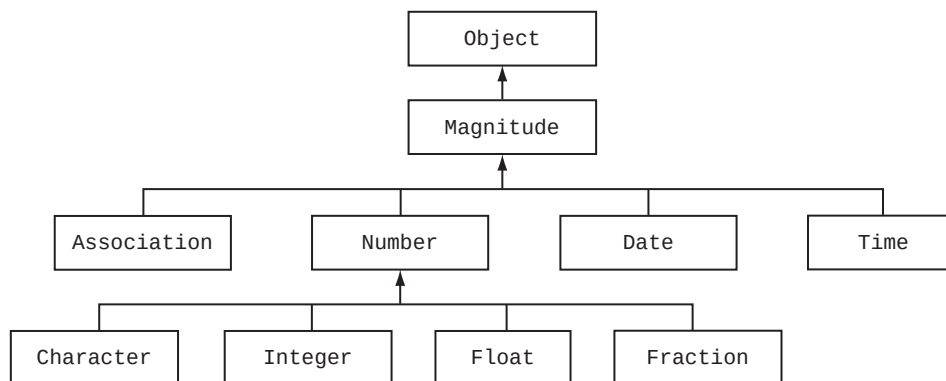


Figure 5.1 Smalltalk's magnitude classes

As you can see from Figure 5.2, the magnitude classes include common types of numbers found in other languages, such as `Integer`, `Float`, and `Fraction`, as well as some other types related to numeric quantities, such as `Time` and `Date`. These classes are sometimes called **concrete classes**,

because their instances are objects that applications normally create and manipulate. By contrast, two other classes in this hierarchy, *Magnitude* and *Number*, are sometimes called **abstract classes**. They are not normally instantiated, but instead serve as repositories of common properties and behaviors for the classes below them in the hierarchy.

We now explore the magnitude class hierarchy in Smalltalk's class browser, to see how classes and their methods are defined and also to learn how the designers of the language already leverage inheritance and polymorphism. (Recall that Smalltalk was conceived not just as a language but also as a programming environment.) Figure 5.2 shows a class browser with the list of magnitude classes visible in the leftmost pane of the window.

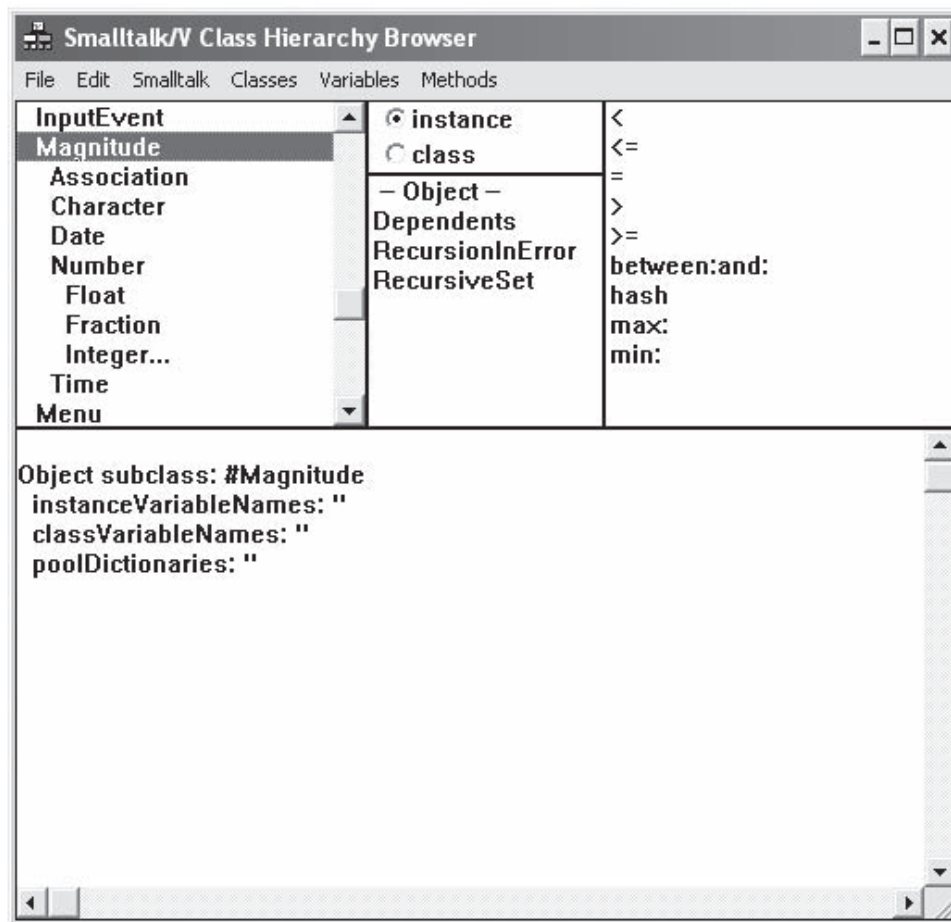


Figure 5.2 A class browser open on the magnitude classes

Note that the most abstract class, *Magnitude*, has been selected. Its instance messages are listed in the rightmost pane. These include the comparison operators and several other messages. These are the most basic messages that any object of a more specific type of magnitude should also recognize. In the bottom pane of Figure 5.2 is code that states that *Magnitude* is a subclass of *Object*; this implies that any message defined in the *Object* class will also be recognized by objects of any magnitude type (or

any other type in Smalltalk). This code also shows that the `Magnitude` class defines no instance variables, class variables, or pool dictionaries, thus leaving it to subclasses to include them if they are needed. Instance variables correspond to storage for properties that belong to each individual object when it is created. Class variables and pool dictionaries correspond to storage for properties that all objects of a class share in common.

Figure 5.3 shows the same browser window after the `max:` message has been selected in the message list.

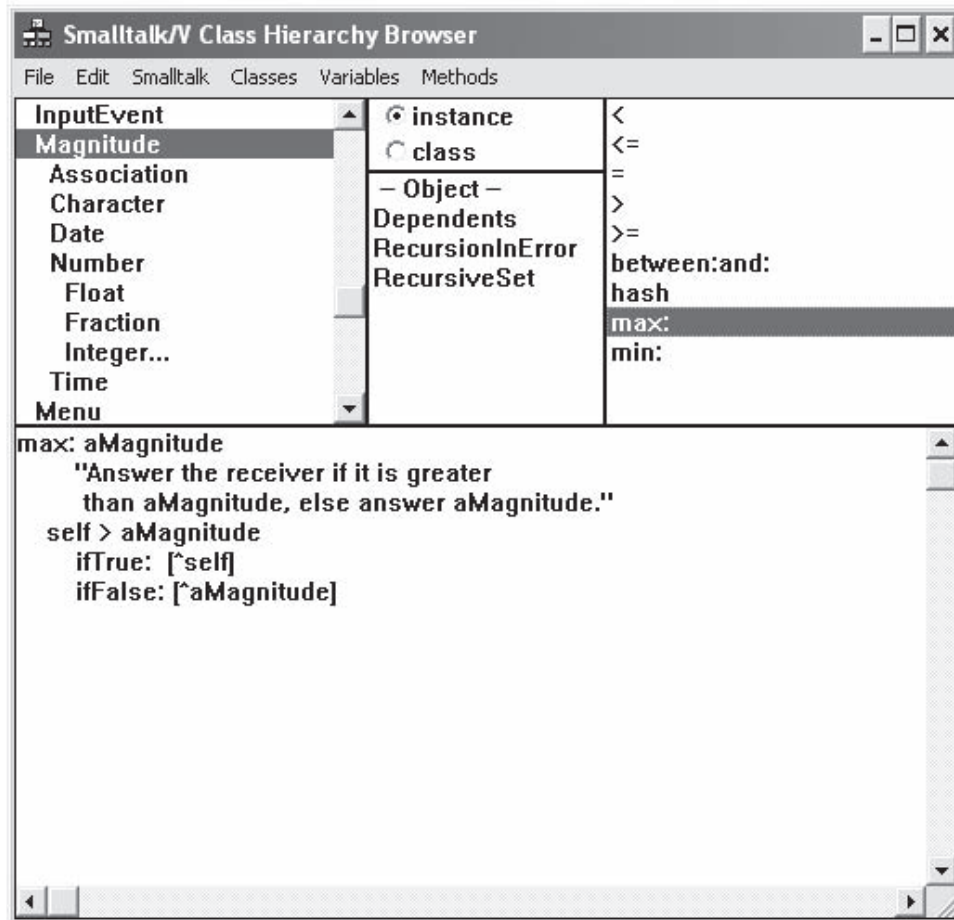


Figure 5.3 The definition of the `max:` method in the `Magnitude` class

When the programmer selects the `max:` message, the method definition corresponding to the message appears in the bottom pane. According to the method header and comment, this method expects another magnitude object as its one argument. The method compares the value of `self`, which refers to the receiver object, to the argument object, and returns the larger of the two objects. (The return operator in Smalltalk is `^`, which has the lowest precedence of any operator.) When referenced in documentation, the methods of a particular class are often prefixed with the class name, to distinguish them from methods with the same name defined in another class. Thus, the method just examined is identified as `Magnitude>>>max:.`

If we then select the > operator in the message list, we see the following code:

```
> aMagnitude
  "Answer true if the receiver is greater
  than aMagnitude, else answer false."
  ^self implementedBySubclass
```

While the code for the implementation of max: is straightforward, the code for the > operator seems unusual, to say the least. It does not tell us how the two magnitudes will be compared, but appears to pass the buck to an unspecified subclass. Nevertheless, most of the power of object-oriented inheritance and polymorphism is exemplified in this code. We now explain why.

The method Magnitude>>>max: is finished and complete; no subclasses, either built-in or new, will ever have to implement it, as long as they implement the > operator. How does this work? When the expression x max: y is run with two fractions, for example, Smalltalk searches first in the class of the receiver object (Fraction) for the method corresponding to this message. Not finding that method in Fraction, Smalltalk moves up to its parent class, Number, where the search fails again. A matching method is finally found in the Magnitude class, which is the parent class of Number. When the method Magnitude>>>max: is then invoked, the symbols self and aMagnitude are in fact still bound to two fraction objects. Consequently, the search process for the method corresponding to the > operator begins with the Fraction class once again. Smalltalk will find the appropriate method definition for comparing two fraction objects in that class, and all will be well.

Before we examine how to define a new concrete magnitude class, let's view some code in the Fraction class for a role model. The code for this class shows, among other things, instance variables for the fraction's numerator and denominator, instance methods that return or modify these values, the > operator for fractions, and a class method to instantiate fractions. The class methods can be viewed by selecting the **Class** radio button in the middle pane of the browser. Here is the code for these resources:

```
Number subclass: #Fraction
  instanceVariableNames:
    'numerator denominator '
  classVariableNames: ''
  poolDictionaries: ''

numerator: n denominator: d
  "(Class method.) Answer an instance of class
  Fraction and initialize the numerator and denominator
  instance variables to n and d respectively."
  ^self basicNew numerator: n denominator: d

numerator: n denominator: d
  "(Instance method.) Answer the receiver.
  The numerator and denominator of the receiver
  are set to the n and d arguments respectively."
```

(continues)

(continued)

```

    numerator := n.
    denominator := d.
    ^self

denominator
    "Answer the denominator of the receiver."
    ^denominator

numerator
    "Answer the numerator of the receiver."
    ^numerator

> aFraction
    "Answer true if the receiver is greater
    than aFraction, else answer false."
    ^(numerator * aFraction denominator) > (denominator * aFraction numerator)

printOn: aStream
    "Append the ASCII representation of
    the receiver to aStream."
    numerator printOn: aStream.
    aStream nextPut: $/.
    denominator printOn: aStream

```

Note that the numerator and the denominator of the receiver object in the `>` method can be accessed directly by mentioning the instance variable names. However, the numerator and denominator of the parameter object can only be accessed by sending it the corresponding messages. Note also that the `>` method in the `Fraction` class uses the `>` operator once again (polymorphically), this time with the numbers that represent the products of the numerators and denominators. The same holds true for the `printOn:` method, which is run again with the numerator and the denominator. Finally, the class includes two methods named `numerator:denominator:`. One of these is a class method that is used to instantiate a `Fraction`. The other is an instance method that resets the values of an existing `Fraction` object's instance variables to new values. In another interesting example of polymorphism, the class method calls the instance method with the same name!

Now we can write code in the transcript window to create two fractions and find the larger of the two:

```

| oneHalf twoThirds |
oneHalf := Fraction numerator: 1 denominator: 2.    "Create two fractions"
twoThirds := Fraction numerator: 2 denominator: 3.
(oneHalf max: twoThirds) printString                "Returns '2/3'"

```

We have examined two definitions of the `>` operator, one abstract (in the `Magnitude` class, with implementation deferred) and the other concrete (in the `Fraction` class, with implementation

realized). They are polymorphic, because they have the same name but are defined differently in different classes. The instance of `>` in the `Magnitude` class is only there to support the use of comparisons in `Magnitude>>>max:`, whereas the instance of `>` in the `Fraction` class is there to realize that behavior specifically for fractions. Likewise, Smalltalk locates the `printString` method for fractions in the `Object` class, and this method in turn sends the `printOn:` message to the fraction object to build its string representation. When a message is sent to an object, Smalltalk binds the message name to the appropriate method found during the search process described earlier. **Dynamic** or **runtime binding** of messages to methods based on the class of the receiver object is probably the most important key to organizing code for reuse in object-oriented systems.

Now that we have seen how Smalltalk's magnitude classes fit together, we are ready to add one of our own. Smalltalk has no built-in class for complex numbers, so we define a new one, `Complex`, as a subclass of `Number`. Doing so will give us the benefit of inheriting all of the implemented methods in `Number` and `Magnitude`, and also require us to implement several of the methods marked as implemented by a subclass. Before we do so, however, let's anticipate some of the behavior of our new class by imagining the following session, which creates two complex numbers and displays their product in the transcript window:

```
| c1 c2 c3 |
c1 := Complex realPart: 3.0 imaginaryPart: 2.5.
c2 := Complex realPart: 6.0 imaginaryPart: 4.5.
c3 := c1 * c2.
Transcript nextPutAll: (c3 printString); cr
69.75+96.0i
```

We start by selecting the `Number` class and then selecting the **Add Subclass** option in the **Classes** menu. This allows the programmer to name the new class and begin entering information about its instance variables in the template shown in the bottom pane. There are two instance variables, `realPart` and `imaginaryPart`. After adding them, we compile the code by selecting menu option **File/Save**. Next, we select the **Class** radio button in the middle pane and the menu option **Methods/New Method** to define a class method to instantiate complex numbers. The code for this method, entered in the method template in the bottom pane, is similar to the code for the class method in the `Fraction` class shown earlier. Next, we define two instance methods with the same names as the instance variables. These methods should return the values of those variables, respectively. As you have seen, users of a Smalltalk object other than `self` cannot access its instance variables without sending a message to that object.

The only job left is to fill in the code for the remaining methods that should be implemented by this class. This can be done incrementally, by adding individual methods, compiling them, and testing them. The complete set of methods is listed in the `Magnitude` and `Number` classes. While there are a daunting number of these methods, many of them are already implemented in terms of more basic operations, such as arithmetic and comparisons. After we provide these, we might be 90% finished. Code reuse provides a big win here!

We conclude this subsection by presenting the code for the properties of the `Complex` class, the types of methods shown earlier for the `Fraction` class, and the `>` and the `*` operators, which first appear in the `Magnitude` and `Number` classes, respectively.

```

Number subclass: #Complex
  instanceVariableNames:
    'realPart imaginaryPart '
  classVariableNames: ''
  poolDictionaries: ''

realPart: r imaginaryPart: i
  "(Class method.) Answer an instance of class Complex
  and initialize the realPart and imaginaryPart
  instance variables to r and i respectively."
  ^self basicNew realPart: r imaginaryPart: i

realPart: r imaginaryPart: i
  "(Instance method.) Answer the receiver. The
  realPart and imaginaryPart of the receiver are
  set to the r and i arguments respectively."
  realPart := r.
  imaginaryPart := i.
  ^self

imaginaryPart
  "Answer the imaginary part."
  ^imaginaryPart

realPart
  "Answer the real part."
  ^realPart

> aNumber
  "Answer true if receiver is greater than aNumber or false otherwise."
  ^(self realPart > aNumber) realPart or: [
    (self realPart = aNumber realPart) and: [
      self imaginaryPart > aNumber imaginaryPart]]

* aNumber
  "Answer the result of multiplying
  the receiver by aNumber."
  | rp ip |
  rp := self realPart * aNumber realPart -
    self imaginaryPart * aNumber imaginaryPart.
  ip := self realPart * aNumber imaginaryPart +
    self imaginaryPart * aNumber realPart.
  ^self class realPart: rp imaginaryPart: ip

```

(continues)

(continued)

```

printOn: aStream
    "Append the ASCII representation of
    the receiver to aStream."
    realPart printOn: aStream.
    aStream nextPut: $+.
    imaginaryPart printOn: aStream.
    aStream nextPut: $i

```

5.2.3 The Collection Hierarchy

Collections are containers whose elements are organized in a specific manner. The different types of organization include linear, sorted, hierarchical, graph, and unordered. Linear collections include the familiar examples of strings, arrays, lists, stacks, and queues. Unordered collections include bags, sets, and dictionaries (also known as map collections in Java). Graphs and trees are also collections, as are sorted collections, whose elements have a natural ordering (each element is less than or equal to its successor).

Built-in collections in imperative languages, such as FORTRAN, C, Pascal, and Ada, have historically been limited to arrays and, if the application programmer is lucky, strings. The programmer must build the other types of collections using the array data structure or by defining linked nodes. Functional languages like Lisp include the list, which is essentially a linked structure, as the basic collection type.

Smalltalk provides the standard for object-oriented languages by including a large set of collection types. They are organized, for maximum code reuse, in a collection class hierarchy. Figure 5.4 shows a class diagram of the most important Smalltalk collection classes.

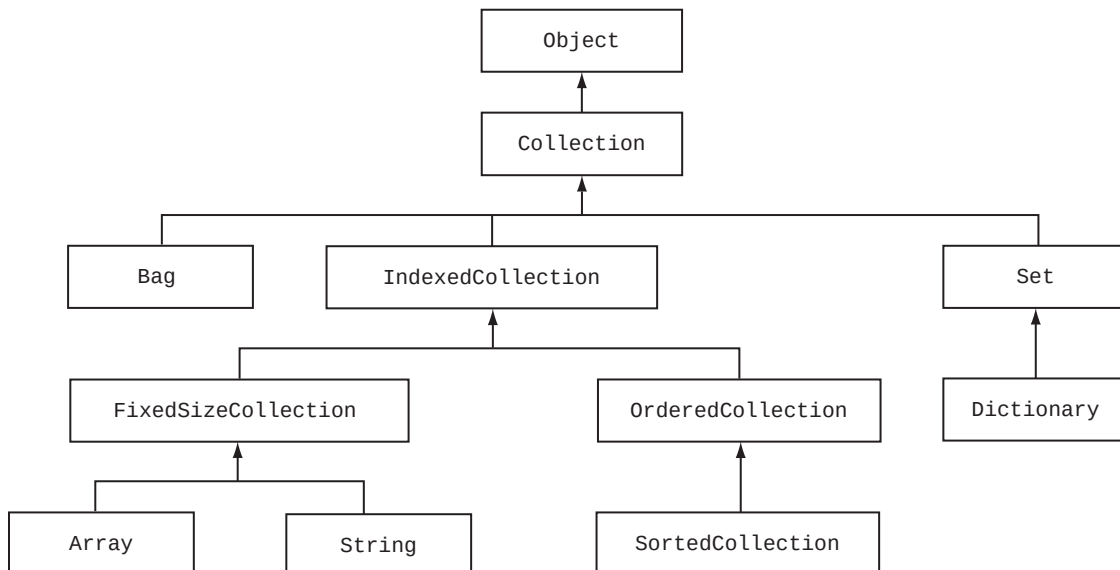


Figure 5.4 The Smalltalk collection class hierarchy

As you have seen in the case of the magnitude hierarchy, the classes in the collection hierarchy are split into abstract classes for organizing common structure and behavior and concrete classes that are instantiated in applications. The most abstract class is `Collection`, which naturally includes methods that should apply to any type of collection. The two most interesting types of methods in this class are the iterators and the methods to convert one type of collection to another type. We now examine the use and implementation of several of these.

The most basic iterator is `do:`. As you saw earlier, this iterator implements a loop that allows the programmer to traverse or visit all of the elements in a collection. Because the manner in which this is done will vary with the organization of the collection, `do:` is implemented by subclasses. However, once the subclass implementation of `do:` is set, the `Collection` class can provide complete implementations of all of the other basic types of iterators, which are listed in Table 5.1.

Table 5.1 The basic types of iterators in Smalltalk's <code>Collection</code> class		
Message	Iterator Type	What It Does
<code>do:aBlock</code>	Simple element-based traversal	Visits each element in the receiver collection and evaluates a block whose argument is each element.
<code>collect:aBlock</code>	Map	Returns a collection of the results of evaluating a block with each element in the receiver collection.
<code>select:aBlock</code>	Filter in	Returns a collection of the objects in the receiver collection that cause a block to return true.
<code>reject:aBlock</code>	Filter out	Returns a collection of the objects in the receiver collection that do not cause a block to return true.
<code>inject:anObject into:aBlock</code>	Reduce or fold	Applies a two-argument block to pairs of objects, starting with <code>anObject</code> and the first element in the collection. Repeatedly evaluates the block with the result of its previous evaluation and the next element in the collection. Returns the result of the last evaluation.

Because they are defined in the `Collection` class, Smalltalk iterators are highly polymorphic. They can work with any types of collections, not just lists, as in functional languages like Lisp. The following series of examples shows the use of the different Smalltalk iterators with strings.

```
'Hi there' collect: [ :ch |
    ch asUpperCase]                                "Returns 'HI THERE'"

'Hi there' select: [ :ch |
    ch asVowel]                                    "Returns 'iee'"

'Hi there' reject: [ :ch |
    ch = Space]                                    "Returns 'Hithere'"

'Hi there' inject: Space into: [ :ch1 :ch2 |
    ch1, ch2]                                       "Returns ' H i t h e r e'"
```

The two iterators `select:` and `inject:into:` can be used to generate the sum of the odd numbers in a collection named `col`, as follows:

```
(col select: [ :n | n odd])
  inject: 0 into: [ :n1 :n2 |
    n1 + n2]                                "Returns the sum of the odd numbers in col"
```

The implementations of Smalltalk iterators rely on the methods `do:` and `add:`, both of which are implemented by subclasses of `Collection`. Here is the code for one of the iterators, `collect:`, which is the polymorphic equivalent of a map in functional languages:

```
collect: aBlock
  "For each element in the receiver, evaluate aBlock with
  that element as the argument. Answer a new collection
  containing the results as its elements from the aBlock
  evaluations."
  | answer |
  answer := self species new.
  self do: [ :element |
    answer add: (aBlock value: element)].
  ^answer
```

Note that this code creates a new instance of the same class as the receiver object. To this new collection object are added the results of evaluating a block with each element of the receiver collection. The new collection is then returned. A similar `do:` loop pattern is used in the implementations of the other iterators.

Another interesting set of methods in the `Collection` class allows the programmer to convert any type of collection, including newly added ones, to many of the built-in types of collections. Most of these methods, such as `asBag` and `asSet`, create an instance of the specified type of collection and send it the `addAll:` message with the receiver object as an argument. The `addAll:` method, which is also included in the `Collection` class, runs a `do:` loop on its parameter object to add each of its elements to the receiver with the receiver's `add:` method. The `add:` method is, naturally, implemented by a subclass. Here is the code for the `Collection` methods `asBag` and `addAll:`.

```
asBag
  "Answer a Bag containing the elements of the receiver."
  ^(Bag new)
    addAll: self;
    yourself

addAll: aCollection
  "Answer aCollection. Add each element of
  aCollection to the elements of the receiver."
  aCollection do: [ :element | self add: element].
  ^aCollection
```

Smalltalk has no built-in classes for stacks, queues, trees, or graphs, but we can easily see how to add them. Let's develop this idea for queues. There are two common implementations of queues, one based on an array and the other based on linked nodes. An array-based queue class could be a subclass of the `Array` class or the `OrderedCollection` class (which is like the array-based `ArrayList` class in Java). However, subclassing one of these classes for queues would be a poor design choice, because users of such queues would then have at their disposal operations, such as `at:put:`, which would violate the spirit of restricting access to the front or rear of a queue collection. A better choice would be to subclass the `Collection` class, whose operations, such as the iterators and the type conversion methods, provide some positive benefits without the negative consequences. The `ArrayQueue` methods would include not just the standard ones such as `enqueue`, `dequeue`, `size`, and `isEmpty` but also `do:` and `add:`, which are needed to invest queues with the functionality of many of the `Collection` methods. As for properties, the `ArrayQueue` class would contain either an array or an ordered collection to hold its data elements. The completion of this version of a queue in Smalltalk is left as an exercise.

The other major implementation, `LinkedList`, is also a subclass of `Collection`. It has the same interface, or set of methods, as `ArrayQueue`. However, the elements in a `LinkedList` are contained in a sequence of linked nodes. Linked nodes come in many flavors, among them singly linked, doubly linked, and variations on each of these with a circular link back to the first node or a special header node. To support any of these options, we need a separate class that contains a data element and one or more links. For a simple singly linked structure, the `Association` class in Smalltalk's magnitude hierarchy already fills the bill.

Two instance variables that point to the first and last nodes in the linked structure enable the implementation to quickly locate the front and rear elements in the queue, respectively. The size of the queue is also tracked with a separate instance variable. Following is a listing of the properties and methods of a `LinkedList` class in Smalltalk:

```
Collection subclass: #LinkedList
  instanceVariableNames:
    'front rear size '
  classVariableNames: ''
  poolDictionaries: ''

  new
    "Answer an instance of the receiver, with front and
    rear pointers as empty links and size of 0."
    ^super new initialize

  initialize
    "Answer this object. Set the instance variables to initial values."
    | newNode |
    front := nil.
    rear := nil.
    size := 0.
    ^self
```

(continues)

(continued)

```

enqueue: anObject
    "Answer anObject. Add anObject to the rear of the receiver."
    | newNode |
    newNode := Association key: anObject value: nil.
    rear notNil ifTrue: [rear value: newNode].
    rear := newNode.
    front isNil ifTrue: [front := newNode].
    size := size + 1.
    ^anObject

dequeue
    "Answer the object at the front of the receiver.
    Also removes this object from the receiver."
    | oldNode |
    oldNode := front.
    front := front value.
    front isNil ifTrue: [rear := nil].
    size := size - 1.
    ^oldNode key

size
    "Answer the size of the receiver."
    ^size

add: anObject
    "A synonym for enqueue."
    ^self enqueue: anObject

do: aBlock
    "Basic iterator, evaluates aBlock on each object
    in the receiver."
    | probe |
    probe := front.
    [probe notNil] whileTrue: [
        aBlock value: probe key.
        probe := probe value]

```

Note that the class method `new` and the instance method `initialize` collaborate to instantiate a `LinkedList`. The method `LinkedList>>>new` sends the message `new` to `super`, which refers to one of `LinkedList`'s superclasses, to obtain a new object. This object then receives the message `initialize`, which sets the instance variables in it. The reason that we need a separate instance method to do this is that the class method `new` cannot access any instance variables.

To examine the behavior and state of a linked queue, we evaluate the following code segment in the transcript window:

```
| q |
q := LinkedList new.
q addAll: #(1 2 3 4 5).      "Copy data from an array literal."
q dequeue.
q enqueue: 'Ken'.
q inspect
```

Smalltalk's `inspect` message opens an inspector window on the receiver object. The programmer can then browse the values of the object's instance variables in this window. Figure 5.5 shows the inspector window for our linked queue.

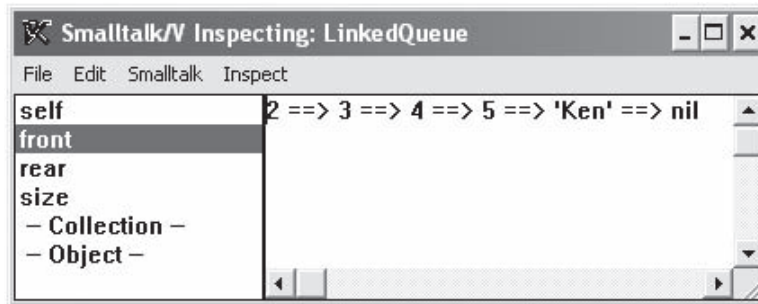


Figure 5.5 A Smalltalk inspector window on a `LinkedList` object

Note that the state of the instance variable `front` actually looks like a linked structure. The programmer can browse into the objects contained in the structure by double-clicking each one.

Smalltalk programmers have found the language and its programming environment a joy to work with. However, in the early 1990s, just as Smalltalk was coming into use on ordinary computers, two languages caught on and stole any thunder that Smalltalk might have built for object-oriented programming. These languages are Java and C++, which we explore in the next two sections.

5.3 Java

James Gosling and his team at Sun Microsystems originally intended Java as a programming language for systems embedded in appliances such as microwave ovens. Their initial emphasis was on portability and a small footprint for code, as captured in the slogan, “write once, run anywhere.” To achieve this goal, Java programs would compile to machine-independent byte code, and each target platform would provide its own Java Virtual Machine or interpreter for this byte code. It turned out that this vision of software development and distribution was ideally suited for many of the developments and advances in computing in the last 20 years, including networks, the Web, and modern developments like handheld, mobile devices.

Java's designers also realized that the language for this sort of application development should be fully object oriented, and, to appeal to the widest range of programmers, have a conventional syntax such

as that of C. Finally, the language should come with a large set of libraries to support applications in all areas of modern computing.

Java's portability, small code footprint, conventional syntax, and support for compile-time type checking gave it an immediate advantage over Smalltalk. Nevertheless, it adopted many of the features of Smalltalk, including runtime garbage collection and the use of references instead of explicit pointers, which make Java a safer language than C or C++. Moreover, unlike C++, Java is not a hybrid of object-oriented and procedural languages, but is purely object oriented, with one exception. In Java, the scalar data types (also called **primitive types**) are not objects, for reasons of efficiency.

In this section, we give an overview of the features of Java that support object-oriented programming. In the process, you will see that the language not only allows the programmer to write Smalltalk code in C-like syntax but also represents some advances on the Smalltalk way of developing programs.

5.3.1 Basic Elements of Java: Classes, Objects, and Methods

As in Smalltalk, a Java application instantiates the appropriate classes and gets the resulting objects to do things by calling methods on them (note the change in terminology, compared to Smalltalk's "sending messages to them"). The only exceptions to this are the use of the scalar types, such as `int`, `float`, and `char`, which appear as operands in expressions or are passed as arguments to methods or returned as their values. Classes such as `String` are already available in the standard `java.lang` package, or may be imported from other packages, such as `java.util` or `javax.swing`. Programmer-defined classes can also be placed in their own packages for import in any applications.

The syntax of variable declaration, object instantiation, and method calls in Java closely resembles that of variable declarations and function calls in C. Variable declaration and object instantiation use the following syntax:

```
<class name> <variable name> = new <class name>(<argument-1, . . . , argument-n>);
```

and a method call, as in Smalltalk, places the object to the left of the method name:

```
<object>.<method name>(<argument-1, . . . , argument-n>)
```

For example, let's assume that someone has defined a new class called `Complex` for complex numbers and placed it in a package called `numbers`. The following short application creates two complex numbers, adds them, and displays the results in the terminal window:

```
import numbers.Complex;

public class TestComplex{

    static public void main(String[] args){
        Complex c1 = new Complex(3.5, 4.6);
        Complex c2 = new Complex(2.0, 2.0);
        Complex sum = c1.add(c2);
        System.out.println(sum);
    }
}
```

This application consists of an import statement, a class definition, and a single method definition within that class. The method is `static`, which means that it is a **class method** and can be run as `TextComplex.main(<array of strings>)`. This is exactly what the Java Virtual Machine does automatically when the application is launched. The JVM places in the `args` array any command-line arguments that might be present at the launch. Our main method ignores this argument and performs as expected.

Now let's develop a small portion of the new `Complex` class for representing complex numbers. Because numbers are not objects in Java, we cannot subclass under a magnitude hierarchy that will give us built-in functionality. Thus, we build the `Complex` class from scratch. Here is a listing with an explanation following:

```
package numbers;

public class Complex extends Object{

    // Default constructor
    public Complex(){
        this(0, 0);
    }

    // Parameterized constructor
    public Complex(double realPart, double imaginaryPart){
        re = realPart;
        im = imaginaryPart;
    }

    public double realPart(){
        return re;
    }

    public double imaginaryPart(){
        return im;
    }

    public Complex add(Complex c){
        return new Complex(this.realPart() + c.realPart(),
                           this.imaginaryPart() + c.imaginaryPart());
    }

    public Complex multiply(Complex c){
        return new Complex(this.realPart() * c.realPart() -
                           this.imaginaryPart() * c.imaginaryPart(),
                           this.realPart() * c.imaginaryPart() +
                           this.imaginaryPart() * c.realPart());
    }
}
```

(continues)

(continued)

```

    public String toString(){
        return this.realPart() + "+" + this.imaginaryPart() + "i";
    }

    // Instance variables
    private double re, im;
}

```

The module begins by declaring that this resource belongs to a Java package named `numbers`. The class header shows that `Complex` extends Java's `Object` class. This extension could have been omitted because it happens by default, but is added for emphasis. The instance variables for `Complex` class are `re` and `im`. They are declared with `private` access, meaning that they are visible and accessible only within the class definition. This is how data encapsulation is enforced in Java. The two **accessor methods** `realPart` and `imaginaryPart` are defined with `public` access. This allows programmers who use complex numbers to view but not modify their internal state. Note that these methods are also called on the receiver object referenced by the keyword `this` and the parameter object referenced by `c`, within the methods `add`, `multiply`, and `toString`. This might seem like overkill, in view of the fact that the instance variables of both objects could have been referenced directly in these methods (using `this.re` or just `re` for the receiver's variable and `c.re` for the parameter's variable). However, the use of the two accessor methods within the class definition places an abstraction barrier between the data and their use, which allows the programmer to change how the data are represented without disturbing other code (more on this point shortly).

The method `toString` is the only polymorphic method defined in the `Complex` class. Its definition overrides the one inherited from the `Object` class. The purpose of `toString` in Java is to return a string representation of the receiver object (in a manner similar to Smalltalk's `printOn:` and `printString`). As defined in `Object`, the default behavior of `toString` is to return the name of the receiver's class. The definition of `toString` in the `Complex` class returns information about the instance, not the class. Some Java methods, such as `System.out.println` and the string concatenation operator `+`, automatically run an object's `toString` method to obtain its text.

In addition to the instance variables and methods, the definition of `Complex` contains two overloaded **constructors**. These are like methods, except that they have the same name as the class and have no return value. Constructors specify initial values for the instance variables and can perform other operations required when an object is first constructed (hence the name). The second constructor has two parameters that provide initial values for the instance variables. The first constructor takes no parameters (it is a so-called **default constructor**) and initializes both `re` and `im` to 0. Note that the first constructor accomplishes its task by calling the second one, using the keyword `this` with the appropriate arguments. This process, called **constructor chaining**, is another form of code reuse.

As mentioned earlier, the use of `private` access to instance variables and the use of accessor methods even within the class definition allow us to change the representation of the data without disturbing other

code. We could rewrite the class `Complex` to use polar coordinates without changing any of the methods, other than `realPart`, `imaginaryPart`, and one constructor, as shown in the following code:

```
package numbers;

public class Complex extends Object{

    public Complex(){
        this(0, 0);
    }

    public Complex(double realPart, double imaginaryPart){
        radius = Math.sqrt(realpart * realpart + imaginaryPart * imaginaryPart);
        angle = Math.atan2(imaginaryPart, realpart);
    }

    public double realPart(){
        return radius * Math.cos(angle);
    }

    public double imaginaryPart(){
        return radius * Math.sin(angle);
    }

    public Complex add(Complex c){
        return new Complex(this.realPart() + c.realPart(),
                           this.imaginaryPart() + c.imaginaryPart());
    }

    public Complex multiply(Complex c){
        return new Complex(this.realPart() * c.realPart() -
                           this.imaginaryPart() * c.imaginaryPart(),
                           this.realPart() * c.imaginaryPart() +
                           this.imaginaryPart() * c.realPart());
    }

    public String toString(){
        return this.realPart() + "i" + this.imaginaryPart();
    }

    private double radius, angle;
}
```

Like Smalltalk, Java uses reference semantics for variables that name objects. Classes are, thus, also called **reference types**. Variables that name values of primitive types, such as `int`, `float`, and `char`, have value semantics in Java. The equality operator `==` used with primitive types can also be used

with reference types, but in that context, it means object identity, as it does in Smalltalk. Java includes an `equals` method in the `Object` class which defaults to the behavior of the `==` operator but can be overridden in subclasses to implement a comparison of two distinct objects for the same properties.

Methods for the objects `z` and `w` can be invoked after instantiation by using the usual dot notation. For example,

```
z = z.add(w);
```

adds `w` to `z`, creating a new `Complex` object in the process, and assigns this new object to `z` (throwing away the object previously pointed to by `z`). It is also possible to nest operations so that, for example,

```
z = z.add(w).multiply(z);
```

performs the computation $z = (z + w) * z$. In the process, a temporary object (we'll call it `t`) is created to hold the value of `z + w`, and a new object is then created to hold the value `t * z`. This object is then referenced by `z`, and `z`'s old object is thrown away. This has essentially all the advantages of abstraction, and with overloading, can make the type `Complex` look more or less like a built-in type. Unfortunately, Java does not allow operator overloading, but C++ does, as discussed later in this chapter.

The disadvantages to this mechanism are two-fold. First, without automatic garbage collection, serious space leaks become difficult to avoid. For this reason, most object-oriented languages require automatic garbage collection (Smalltalk and Java, for example, but not C++). Second, this view of binary operations is not symmetric, since the left-hand operand is singled out as the target of the method call. In a mixed-paradigm language like C++, such operators could be implemented instead as ordinary functions or as overloaded operators in method definitions. Alternatively, some object-oriented languages, such as the Common Lisp Object System (CLOS), allow **multimethods**, in which more than one object can be the target of a method call.

5.3.2 The Java Collection Framework: Interfaces, Inheritance, and Polymorphism

As mentioned earlier, Java includes a large number of libraries or packages with classes to support many kinds of applications. Some of these sets of classes are related in what one might call a framework. One such framework, which is similar in many ways to the Smalltalk collection hierarchy, is the Java collection framework, as defined in the package `java.util`. In this section, we explore Java's support in this framework for inheritance, polymorphism, and distinguishing concrete and abstract classes. We also examine how two other features not found in Smalltalk—parameterized types and interfaces—support the organization of code in a statically typed object-oriented language like Java.

Conceptually, an **interface** is a set of operations on objects of a given type. Interfaces serve as the glue that connects components in software systems. A resource's interface should give its users only the information they need to use it and its implementers only enough information to implement it. Java codifies the concept of an interface with a special syntax that allows the programmer to list a type name and a set of public method headers. Java interfaces can belong to packages and can be compiled in advance of any other code. An interface can then be realized or implemented by any class that includes all of the methods specified in that interface.

Java's collection framework is structured at the top level in terms of a set of interfaces. Figure 5.6 shows a diagram of some of them.

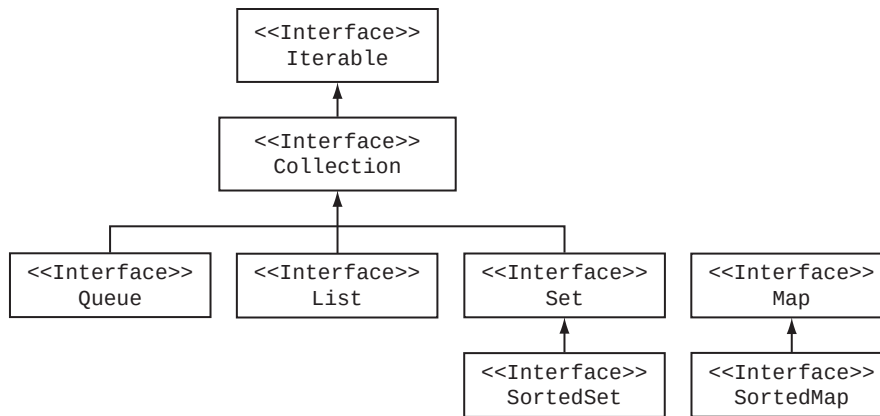


Figure 5.6 Some Java collection interfaces

The interfaces appear in a tree-like hierarchy, much like classes. The more general interfaces are at the top. Each interface below the root extends its parent interface above it. Thus, for example, the `List` interface includes not only the methods specific to lists but also implicitly those in the `Collection` and `Iterable` interfaces. The position of the `Iterable` interface at the root of this tree ensures that every collection must include an iterator method. We will have much more to say about iterators in a moment. Java's designers thought it best not to include the `Map` interfaces in the hierarchy (see Exercise 5.8).

The next code segment shows the code for some of the methods specified in the `Collection` interface:

```

public Interface Collection<E> extends Iterable<E>{
    boolean    add(E element);
    boolean    addAll(Collection<E> c);
    void       clear();
    boolean    contains(E element);
    boolean    containsAll(Collection<E> c);
    boolean    equals(Object o);
    boolean    isEmpty();
    boolean    remove(Element E);
    boolean    removeAll(Collection<E> c);
    boolean    retainAll(Collection<E> c);
    int        size();
}

```

You will recall some similar methods from our discussion of Smalltalk's `Collection` class. Note also the presence of notations such as `<E>` and `E` in the interface and method headers. These notations represent **type parameters**. You saw type parameters used with the functional language Haskell in Chapter 4. Java is statically typed, and all data types, including the types of the elements in collections, must be explicitly specified at compile time. Collections with type parameters are also

called **generic collections**, as distinct from the nongeneric, **raw collections** that appeared in early versions of Java. A generic collection exploits parametric polymorphism, which allows collections of different element types to rely on a single definition of the collection type.

Java collections can contain elements of any reference types. The complete type (including the actual element type) of a given collection variable must be specified when that variable is declared. For example, the following code declares two variables, one to reference a list of strings and the other to reference a set of integers:

```
List<String> listOfStrings;  
Set<Integer> setOfInts;
```

Note that the type parameters for the element types of `List` and `Set` have been replaced by the actual element type names, `String` and `Integer`, respectively. This prevents the programmer from adding anything other than a string to the list and anything other than an integer to the set. (Java automatically wraps `int` values in `Integer` objects when they are added to a collection of integers, and automatically unwraps them when they are returned from a collection.)

The classes that implement the collection interfaces also form a hierarchy. Figure 5.7 shows some of these. The dashed arrow in the diagram indicates that a class implements an interface. A solid arrow indicates that a class extends another class and an interface extends another interface. If a class implements an interface, any of its subclasses also (implicitly) implement that interface.

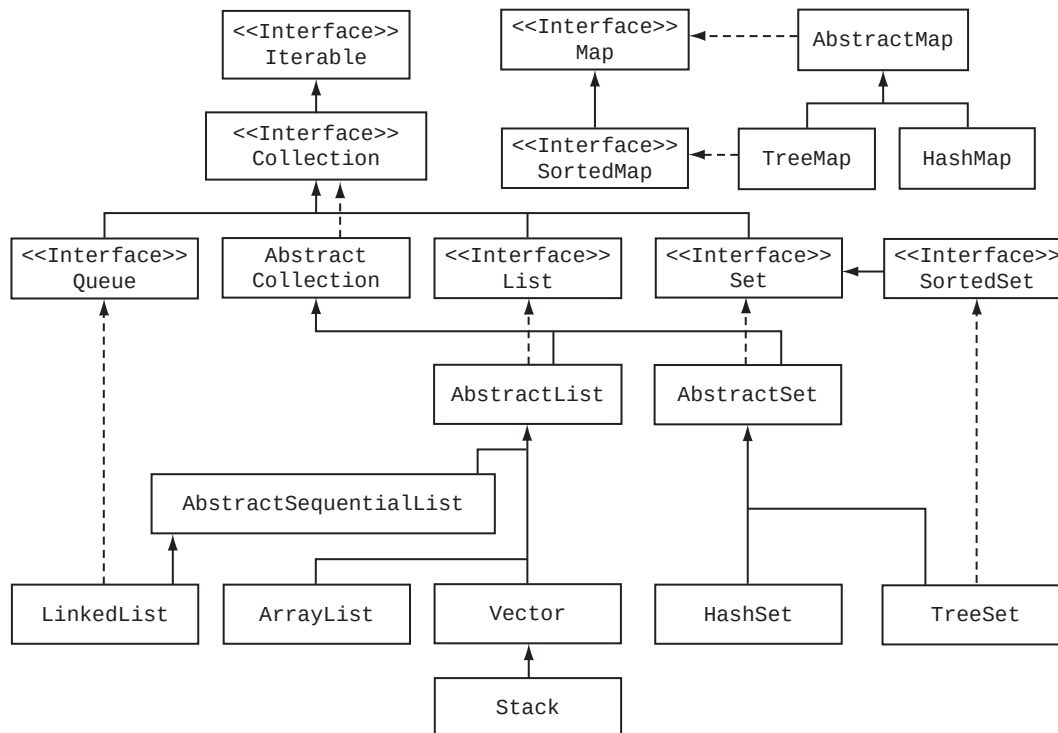


Figure 5.7 Some Java collection classes and their interfaces

Some of these classes are obviously abstract, even without the hints offered by their names. These classes implement common properties and behavior for their subclasses. The classes on the frontier of the tree are concrete and thus can be instantiated. Note also that some classes, such as `LinkedList`, implement more than one interface. Thus, a linked list can behave either as a list or as a queue. For example, the following code segment declares variables for a list of strings, a queue of floats, and a list of characters. The code also initializes each of the variables to new list objects. The first two are linked lists, and the third is an array list.

```
List<String>    listOfStrings = new LinkedList<String>();
Queue<Float>    queueOfFloats = new LinkedList<Float>();
List<Character> listOfChars   = new ArrayList<Character>();
```

From the compiler's (or the programmer's) perspective, the lists of strings and characters in this code are just lists. That is, the same methods can be called on the two `List` variables even though the two lists they reference have different implementations and different element types. However, because the variable `queueOfFloats` has been declared to be of type `Queue<Float>`, the linked list that it refers to will only respond to the methods specified in the `Queue` interface. Any other `List` method used with this variable will be flagged as a syntax error. Thus, the danger that a list object might violate the spirit of a queue, which we saw earlier in our discussion of inheritance in Smalltalk, is avoided with these declarations.

By contrast, the design decision to make the Java `Stack` class a subclass of `Vector`, which implements the `List` interface (see Figure 5.7), is not the most fortunate one. In this case, the spirit of a stack could be violated, because a programmer might use any of the list operations on it.

To remedy this design flaw in Java, let's develop a new type of stack class called `LinkedStack`. This implementation of a stack will behave like a pure, restricted stack, but allow users to run some of Java's `Collection` methods with stacks as well. Our new class will include the standard stack methods `push`, `pop`, and `peek`, as well as the `Collection` methods `size`, `add`, and `iterator`. By defining `LinkedStack` as a subclass of the `AbstractCollection` class, we will get a good deal of additional behavior for free. Figure 5.8 shows a diagram of our proposed subframework.

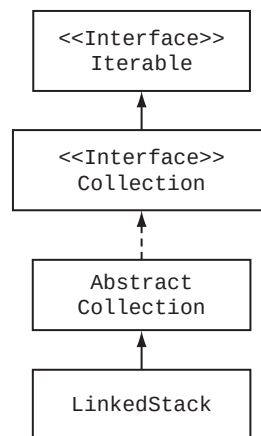


Figure 5.8 Adding `LinkedStack` to the collection framework

Our implementation of `LinkedStack` closely resembles that of our `LinkedQueue` class in Smalltalk. We use a singly linked sequence of nodes to hold the elements, with a single external pointer, called `top`, that points to the first node. An integer used to track the number of elements, named `size`, is the only other instance variable. The `Node` class is not defined externally but is nested as a **private inner class** within the `LinkedStack` class. We postpone the development of the iterator for stacks for a moment. Here is the code for the basic `LinkedStack` class:

```
import java.util.AbstractCollection;
import java.util.Iterator;

public class LinkedStack<E> extends AbstractCollection<E>{

    // Constructor
    public LinkedStack(){
        top = null;
        size = 0;
    }

    public E peek(){
        return top.data;
    }

    public E pop(){
        E data = top.data;
        top = top.next;
        size -= 1;
        return data;
    }

    public void push(E data){
        top = new Node<E>(data, top);
        size += 1;
    }

    // The next three methods are required by AbstractCollection
    public boolean add(E data){
        push(data);
        return true;
    }

    public int size(){
        return size;
    }
}
```

(continues)

(continued)

```

    // Just a stub for now; will be completed shortly
    public Iterator<E> iterator(){
        return null;
    }

    // Instance variables for LinkedStack
    private Node<E> top;
    private int size;
    // Node class for LinkedStack
    private class Node<E>{

        // Constructor for Node
        private Node(E data, Node<E> next){
            this.data = data;
            this.next = next;
        }

        // Instance variables for Node
        private E data;
        private Node<E> next;
    }
}

```

The header of the `LinkedStack` class includes the type variable notation `<E>` after both `LinkedStack` and `AbstractCollection` (as explained shortly, the return type of the `iterator` method, `Iterator<E>`). The nested `Node` class uses the same notation for the element type. Likewise, all uses of the `Node` type in variable declarations must include the same notation. The type variable `E` is used directly without the angle brackets for the type of elements in variable and parameter declarations and for the return type of methods.

The nested `Node` class is defined as `private`, because no classes outside of `LinkedStack` need to use it. Its constructor and instance variables are also defined as `private`. The `Node` class includes no accessor or mutator methods for its `data` and `next` fields; it is intended as a simple container class whose fields can safely be referenced directly using the dot notation within the `LinkedStack` implementation. Thus, `Node` includes just a constructor and two instance variables. The `Node` class is recursive; one of its instance variables, a reference to the next node, is also of type `Node<E>`. Java has no explicit pointer type, so links are always implemented as references. The `null` value, which indicates an empty link, can be assigned to a variable of any reference type.

Before we develop the `iterator` method for `LinkedStack`, a few words must be said about Java iterators. The `iterator` method is the only method specified in the `java.util.Iterable` interface. This method returns a new object of type `Iterator` when run on a collection object. The collection object is also called the **backing store** for the iterator object. For example, one could create a `LinkedStack` of strings and then open an iterator object on it as follows:

```

LinkedStack<String> stack = new LinkedStack<String>();
Iterator<String> iter = stack.iterator();

```


The relationship between the stack and iterator objects is shown in Figure 5.9.

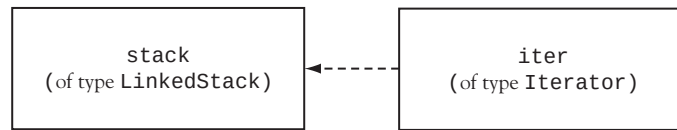


Figure 5.9 An iterator opened on a stack object

Note that the `Iterator` type is also parameterized for the same element type as its backing store.

The `Iterator` type is an interface, which specifies the three methods shown in the following code segment:

```
public Interface Iterator<E>{

    public boolean hasNext(); // True if more elements, false otherwise
    public E next();          // Returns the current element and advances
    public void remove();     // Removes the current element from the store
}
```

Returning to our earlier code segment, where we created an iterator object named `iter`, we can use the `Iterator` methods `hasNext` and `next` in a `while` loop to visit each of the elements in the iterator's backing store (the linked stack). The next code segment simply prints these elements:

```
while (iter.hasNext())
    System.out.println(iter.next());
```

Java's enhanced `for` loop is syntactic sugar for the iterator-based `while` loop just shown. Although the next code segment appears to loop through the stack, the code using the iterator with a `while` loop is actually at work behind the scene:

```
for (String s : stack)
    System.out.println(s);
```

The only prerequisites for the use of an enhanced `for` loop are that the collection class implements the `Iterable` interface and that it also implements an iterator method.

To implement an iterator method, the `LinkedStack` class must define and instantiate a class that implements the `Iterator` interface. We name this class `StackIterator` and instantiate it in the iterator method, as follows:

```
public Iterator<E> iterator(){
    return new StackIterator<E>(top);
}
```

Note that the `StackIterator` object receives a reference to the top of the stack as an argument when it is created.

The `StackIterator` class is another nested, private class. It maintains a single instance variable called `probe`, which will be used to visit each node in the stack's linked structure. This variable is

initially set to `top`, which was passed to the `StackIterator` object at instantiation (note that `top` and `probe` now refer to the same object, or `null` if the stack is empty). The method `hasNext` returns `false` when `probe` is `null`. This indicates that the end of the stack has been reached. The method `next` saves a reference to the element in the current node, advances the `probe` reference to the next node, and returns the element. The method `remove` must also be included in the `StackIterator` class but should not be supported for stacks. Therefore, this method throws an `UnsupportedOperationException`. Here is the code for the `StackIterator` class:

```
private class StackIterator<E> implements Iterator<E>{

    // Constructor
    private StackIterator(Node<E> top){
        probe = top;
    }

    public E next(){
        E data = probe.data;
        probe = probe.next;
        return data;
    }

    public boolean hasNext(){
        return probe != null;
    }

    public void remove(){
        throw new UnsupportedOperationException();
    }

    // Instance variable
    private Node<E> probe;
}
```

We claimed earlier that interfaces form the glue that holds software components together. In Java, the interfaces `Iterable` and `Iterator` collaborate to support the use of a simple `for` loop over the elements of any collection class. Recall that in `Smalltalk`, the methods `do:`, `size`, and `add:` are implemented by all `Collection` subclasses. These classes then inherit from `Collection` a number of high-level methods, such as `addAll:` and other types of iterators. The Java collection framework is set up to provide similar benefits. If a collection extends `AbstractCollection`, it must implement the methods `iterator`, `size`, and `add`. These methods are declared as `abstract` in the `AbstractCollection` class, meaning that their implementation is deferred to (and required at compile time by) subclasses. However, they are also used in `AbstractCollection` to implement

the other `Collection` methods, such as `addAll` and `contains` (similar to Smalltalk's `includes`). The following snippet of the `AbstractCollection` class shows how this is done:

```
abstract public class AbstractCollection<E> implements Collection<E>{

    // Methods to be implemented by subclasses
    abstract public int size();
    abstract public boolean add(E element);
    abstract public Iterator<E> iterator();

    // Methods implemented here but inherited by subclasses
    public boolean addAll(Collection<E> c){
        boolean added = true;
        for (E element : c)
            added = this.add(element) && added;
        return added;
    }

    public boolean contains(Object target){
        for (E element : this)
            if (! target.equals(element)) return false;
        return true;
    }

    // Other methods
}
```

As you can see, the programmer can use not only a `for` loop with a linked stack but also the `Collection` method `addAll` to copy elements from a list or a set to a linked stack, the `Collection` method `contains` to search a linked stack for a given element, and so forth. The collection hierarchy does not include built-in `map`, `filter`, and `reducing` iterators, but we show how to define these later in this chapter.

5.3.3 Dynamic Binding and Static Binding

As you have seen, Java is pure object-oriented language with a C-like syntax and static typing. Part of Java's success in the programming community, as compared with Smalltalk, is attributable to the latter two qualities. Additionally, the design of Java can allow some of the runtime cost (and most of the error recovery) of method lookup to be shifted from runtime to compile time. The Java compiler statically determines which implementation of a method to use when the receiver object's actual class can be determined at compile time. At that point, all of the information about the class's methods and those of its ancestor classes is available. For example, for the method call:

```
aCollection.addAll(aCollection)
```

the Java compiler can determine that the method `addAll` is implemented in the `AbstractCollection` class, and thus generate the code for a call of that particular implementation of the method. This process

is known as **static binding**, in contrast to the dynamic binding that you saw implemented in Smalltalk. Unfortunately, the Java compiler will not actually generate code for static binding unless the method being called has been defined as `final` or `static`. A final method is one that cannot be overridden in a subclass, and a static method is one that is called on a class rather than an instance. In both of these cases, there can be no possible call down to the class of the receiver object, so the method calls are statically bound.

By contrast, when the compiler gets to the call of `this.add(element)` in the `addAll` method’s code, it cannot determine, at compile time, which concrete version of the `add` method to call. The symbol `this` could refer to a stack, a list, or any other subtype of `Collection`. So, the actual type of this object cannot be determined until runtime. In general, calls “from above” to methods implemented by subclasses must always be handled by dynamic binding at runtime.

Although the residue of dynamic binding in Java incurs a runtime performance hit, Java’s implementation of method lookup, which uses a jump table rather than a search of an inheritance tree, still gives Java an efficiency edge over Smalltalk. Moreover, the compile-time requirement that abstract methods must be implemented by subclasses introduces an aspect of safety (compile-time error checking) that is absent from Smalltalk. In this example, the compiler guarantees that at least there will be an `add` method to call at runtime, once the JVM has determined the actual type of its receiver object.

5.3.4 Defining Map, Filter, and Reduce in Java

Map, filter, and reduce are higher-order functions in a functional language, and built-in collection methods in Smalltalk. As you saw in Section 5.2, these Smalltalk methods are also polymorphic, in that they operate on collections of any type, not just lists. Although Java has only a basic iterator, it is possible to define map, filter, and reduce methods that are similar in behavior to their Smalltalk counterparts. Doing so will serve to illustrate some important points about interfaces, classes, generics, and static typing in Java.

The map, filter, and reduce methods can be defined as static methods in a special class named `Iterators` (similar to Java’s `Math` and `Collections` classes). The map and filter methods expect an input collection as an argument and return an output collection as a value. The programmer can pass to these methods any collection parameter that implements the `Collection` interface. The output collection is also defined to be of type `Collection`, but the actual object returned will be of the same concrete class (with perhaps a different element type) as the input collection. Thus, if we choose to convert a list (or set) of integers to a list of strings, we can send in any list (or set) of integers and get back a list (or set) of strings of the same concrete class (for example, `ArrayList` or `HashSet`), masquerading as a collection. This collection then can be cast down to its actual class if desired. Likewise, the reduce method takes a collection as an argument, but returns an object of the collection’s element type as its result. Table 5.2 shows the rough syntactic form of each method. Note that the element type parameters of the collections are specified, but the exact syntax of the operation parameter is yet to be determined.

Table 5.2 The rough syntax of map, filter, and reduce
<code>public static<E, R> Collection<R> map(<an operation>, Collection<E> c)</code>
<code>public static<E> Collection<E> filter(<an operation>, Collection<E> c)</code>
<code>public static<E> E reduce(E baseElement, <an operation>, Collection<E> c)</code>

The only problem still to be solved is how to represent the operation that is passed as the remaining argument to these methods. In a functional language, this operation is a function of one or two arguments, either predefined or constructed on the fly as a lambda form. In Smalltalk, this operation is a block of code with one or two arguments. In Java, we can define the operation as a special type of object that recognizes a method that will be called within the higher-order method's implementation.

There are actually three types of operations:

1. As used in `map`, the operation is a method of one argument that transforms a collection element into another value (perhaps of a different type). Let's call this operation `R transform(E element)` and include it in a special interface called `MapStrategy`.
2. As used in `filter`, the operation is a method of one argument that returns a Boolean value. We call this method `boolean accept(E element)` and include it in the interface `FilterStrategy`.
3. As used in `reduce`, the operation is a method of two arguments that returns an object of the same type. We call this method `E combine(E first, E second)` and include it in the interface `ReduceStrategy`.

Here is the code for the three interfaces that specify the three types of operations for any mapping, filtering, or reducing process:

```
package iterators;

public interface MapStrategy<E, R>{
    // Transforms an element of type E into a result of type R
    public R transform(E element);
}

package iterators;

public interface FilterStrategy<E>{
    // Returns true if the element is accepted or false otherwise.
    public boolean accept(E element);
}

package iterators;

public interface ReduceStrategy<E>{
    // Combines two elements of type E into a result of type E.
    public E combine(E first, E second);
}
```

The method headers in Table 5.2 can now be completed by including the type of the strategy/object passed to each method. Table 5.3 shows these changes.

Table 5.3 The finished syntax of map, filter, and reduce
<code>public static<E, R> Collection<R> map(MapStrategy<E, R> strategy, Collection<E> c)</code>
<code>public static<E> Collection<E> filter(FilterStrategy< E> strategy, Collection<E> c)</code>
<code>public static<E> E reduce(E baseElement, ReduceStrategy<E> strategy, Collection<E> c)</code>

The use of these three strategy interfaces tells the implementer of the iterators that she can expect an object that recognizes the appropriate strategy method (transform, accept, or combine). Likewise, they tell the user of the iterators that he need only supply an object of a class that implements one of these interfaces to use them correctly. We now present the implementation of the iterators, followed by an example application.

As mentioned earlier, the home of the three iterator methods is a new class named `Iterators`. The simplest method is `reduce`, which combines a collection of objects into a single object. Here is the code:

```
public static<E> E reduce(E baseElement,
                        ReduceStrategy<E> strategy,
                        Collection <E> c){
    for (E element : c)
        baseElement = strategy.combine(baseElement, element);
    return baseElement;
}
```

As you can see, the implementer has no knowledge of the particular operation being used to combine the results, other than that its target object (here named `strategy`) implements the `ReduceStrategy` interface. In this respect, the method call of `strategy.combine` is like the call of a formal function parameter in a higher-order function.

The `map` method is slightly more complicated, because it must create a new collection of the same type as the input collection. Fortunately, the Java methods `getClass()` and `newInstance()` come to our rescue. The first method returns the class of an object, and the second method returns a new instance of this class, using the class's default constructor. The code must be embedded in Java's try/catch exception-handling syntax for the error checking required by these methods. The code for `map` follows. The implementation of `filter` is left as an exercise.

```
public static<E, R> Collection<R> map(MapStrategy<E, R> strategy,
                                    Collection<E> c){
    try{
        Collection<R> results = c.getClass().newInstance();
        for (E element : c)
```

(continues)

(continued)

```
        results.add(strategy.transform(element));
    }
    return results;
} catch (Exception e) {
    System.out.println(e);
    return null;
}
}
```

As mentioned earlier, the user of one of our new iterators is responsible for supplying a collection and an instance of the corresponding strategy interface. The programmer could define a new class that implements MapStrategy, FilterStrategy, or ReduceStrategy, but there is no need to go to that trouble. Java allows the programmer to create an instance of an anonymous class at the point of use, in much the same manner as functional programmers create lambda forms. The syntax for this is:

```
new <interface name>(){
    <method implementations>
}
```

Table 5.4 shows the MapStrategy interface and an example instantiation. The instantiation transforms an integer value into a double value by taking the square root of the integer.

Table 5.4 The MapStrategy interface and an instantiation	
Interface	Instantiation
public interface MapStrategy<E, R>{ public R transform(E element); }	new MapStrategy<Integer, Double>(){ public Double transform(Integer i){ return Math.sqrt(i); } }

The next code segment shows a complete application that takes our new iterators for a test drive. Note that two different particular mapping strategies are used, and then one filtering strategy and one reducing strategy. The same code could be used on a set of integers or on any other collection class (like the LinkedList explored in Section 5.3.2) of integers.

```
import iterators.*;
import java.util.*;

/*
Tests the map, filter, and reduce iterators with array lists.
Creates strategy objects as anonymous instances of the appropriate interfaces.
*/

public class TestIterators{

    static public void main(String[] args){
```

(continues)

(continued)

```
// The list contains [1 2 3 4 5 6 7 8]
List<Integer> list = new ArrayList<Integer>();
for (int i = 1; i <= 8; i++)
    list.add(i);

// Compute the square roots
Collection<Double> squareRoots =
    Iterators.map(new MapStrategy<Integer, Double>(){
        public Double transform(Integer i){
            return Math.sqrt(i);
        }
    }, list);
System.out.println(squareRoots);

// Convert to string representations
Collection<String> stringReps =
    Iterators.map(new MapStrategy<Integer, String>(){
        public String transform(Integer i){
            return i + "";
        }
    }, list);
System.out.println(stringReps);

// Keep the even numbers
Collection<Integer> evens =
    Iterators.filter(new FilterStrategy<Integer>(){
        public boolean accept(Integer i){
            return i % 2 == 0;
        }
    }, list);
System.out.println(evens);

// Sum the numbers
int sum = Iterators.reduce(0, new ReduceStrategy<Integer>(){
    public Integer combine(Integer first, Integer second){
        return first + second;
    }
}, list);
System.out.println(sum);
}
```


How do our Java versions of `map`, `filter`, and `reduce` stack up against their counterparts in functional languages and in Smalltalk? The functional versions are simple, because they are functions that accept other functions as arguments. There is nothing new to learn here. However, `map`, `filter`, and `reduce` are limited to list collections. The Smalltalk versions are more general, because they are polymorphic over any collections. The syntax of a Smalltalk block is really no more complicated than that of a lambda form. The syntax of the Java versions is slightly more complicated, but much of that complexity can be viewed as boilerplate for the essential equivalent of a lambda form or anonymous block of code. The real benefit of the Java versions, as compared to those of Smalltalk, lies in the static type checking. As always, type errors (the use of a noncollection object or a nonstrategy object) are caught early, at compile time, and all the errors are always caught. This makes the use of `map`, `filter`, and `reduce` virtually foolproof, if a bit cumbersome, in Java.

5.4 C++

C++ was originally developed by Bjarne Stroustrup at AT&T Bell Labs (where C was developed) as an extension of C with Simula-like classes, but in the late 1980s and 1990s it grew well beyond its origins into a language containing a large number of features, some object oriented, some not. It is a compromise language in that it contains most of the C language as a subset, as Simula contains Algol60 as a subset. C++ was also designed to be an efficient, practical language, developing out of the needs of its users. As such it became the primary object-oriented language of the early 1990s and remains extremely popular for non-Web applications. Since the first reference manual was published in 1986, many new features have been added, including multiple inheritance, templates, operator overloading, and exceptions. The language has now stabilized since the adoption of an international standard in 1998 (C++98) and as revised in 2003 (C++03). Work is in progress on a new standard (C++0x), whose publication is due near the time this book will be printed.

5.4.1 Basic Elements of C++: Classes, Data Members, and Member Functions

C++ contains class and object declarations similar to those of Java. Instance variables and methods are both called **members** of a class: Instance variables are referred to as **data members**, and methods are referred to as **member functions**. Subclasses are referred to as **derived classes** and superclasses are called **base classes**. One basic difference between C++ and most other object-oriented languages, including Java, is that objects are not automatically pointers or references. Indeed, except for inheritance and dynamic binding of member functions, the `class` data type in C++ is essentially identical to the `struct` (or record) data type of C.

Similar to Java, three levels of protection are provided for class members in C++: `public`, `private`, and `protected`. **Public** members are those accessible to client code, as well as to derived classes. **Protected** members are inaccessible to client code but are still accessible to derived classes. **Private** members are accessible neither to clients nor to derived classes. In a `class` declaration, the default protection is `private`, while in a `struct` declaration, the default is `public`. Additionally, the keywords `private`, `public`, and `protected` establish blocks in class declarations, rather than apply only to individual member declarations as in Java:

```

class A{

// A C++ class

    public:

        // all public members here

    protected:

        // all protected members here

    private:

        // all private members here

};

```

(Note the final semicolon after the closing bracket, which is a vestige of C syntax; a list of variable names could appear before this semicolon, with the preceding class definition as their data type.)

In C++, initialization of objects is given as in Java by **constructors** that have the same name as that of the class. Constructors are automatically called when an object is allocated space in memory, and actual parameters to the constructor are evaluated at that time. Thus, constructors are similar to Java, except that in C++ objects need not be references, and so a constructor can be automatically called as part of a declaration, as well as in a new expression. Unlike Java, C++ does not have required built-in garbage collection, and so C++ also includes **destructors**, which like constructors have the same name as the class, but which are preceded by the tilde symbol “~” (sometimes used in logic to mean “not”). Destructors are automatically called when an object is deallocated (either by going out of scope or through the use of the delete operator), but they must be provided by the programmer and so represent a manual approach to memory deallocation.

A class declaration in C++ does not always contain the implementation code for all member functions. Member functions can be implemented outside the declaration by using the **scope resolution operator** indicated by a double colon “:” after the class name. It gets its name from the fact that it causes the scope of the class declaration to be reentered. Member functions that are given implementations in a class declaration are automatically assumed to be **inline** (that is, a compiler may replace the function call by the actual code for the function).

As a first example in C++, we offer a partial definition of a complex number class:

```

class Complex{

    public:

        // Constructor
        Complex(double r = 0, double i = 0)
        : re(r), im(i) { }

```

(continues)

(continued)

```

    double imaginarypart(){
        return im;
    }

    Complex operator-(){ // unary minus
        return Complex( -re, -im);
    }

    Complex operator+(Complex y){
        return Complex(re + y.realpart(),
                        im + y.imaginarypart());
    }

    Complex operator-(Complex y){ // subtraction
        return Complex(re - y.realpart(),
                        im - y.imaginarypart());
    }

    Complex operator*(Complex y){
        return Complex(re * y.realpart() - im * y.imaginarypart(),
                        im * y.realpart() + re * y.imaginarypart());
    }

    private:

    double re, im;
};

```

This example uses operator overloading in C++, so that complex arithmetic can be written exactly like built-in arithmetic, for example:

```

Complex z(1,2);
Complex w(-1,1);
Complex x = z + w;

```

The syntax for instance variable initialization in a C++ constructor is somewhat unusual. The instance variable names are listed after a colon in a comma-separated list between the constructor declaration and body, with the initial values in parentheses after each instance name. The reason for this notation in C++ is efficiency: C++ requires that all instance variables be initialized *prior* to the execution of the body of any constructor. Thus, initialization would be performed *twice* if it were expressed as regular code inside the body of the constructor (once by the system and once by the actual code). That is, the Complex constructor declaration in this example:

```

Complex(double r = 0, double i = 0)
: re(r), im(i) { }

```

means that `re` is initialized to the value of `r` and `im` is initialized to the value of `i`. In other words, the above code has essentially the same meaning as:

```
Complex(double r = 0, double i = 0){
    re = r; im = i;
}
```

The constructor is also defined with default values given for its parameters:

```
Complex(double r = 0, double i = 0)
```

This allows `Complex` objects to be created with 0, 1, or 2 parameters, and avoids the need to create different overloaded constructors:

```
Complex i(0,1); // i constructed as (0,1)
Complex y; // y constructed as default (0,0)
Complex x(1); // x constructed as (1,0)
```

5.4.2 Using a Template Class to Implement a Generic Collection

You saw how a generic collection class is defined in Java in Section 5.3.2. A similar feature, called **template classes**, is used to define generic collections in C++. Although the standard template library (STL) of C++ includes several built-in collection classes, let's develop a `LinkedList` class to see how a C++ template class works.

Our C++ `LinkedList` class has the same basic interface as the Java version discussed in Section 5.3.2. Here is a short C++ program that exercises some of its capabilities:

```
#include <iostream>
using std::cout;
using std::endl;
#include "LinkedList.c"

int main(){
    int number;
    LinkedList<int> stack;
    for (number = 1; number <= 5; number++)
        stack.push(number);
    cout << "The size is " << stack.size() << endl;
    while (stack.size() != 0)
        cout << stack.pop() << " ";
    cout << endl;
}
```

As you can see, this code declares a stack type in a similar manner to the Java version, although in this case the `int` type can be specified directly as the stack's element type. However, unlike in Java, a new `LinkedList` object is automatically created and assigned to the `stack` variable upon the mere mention of that variable in the declaration. Otherwise, the usage of `LinkedList` is exactly the same as in the Java example.

Our Java version of `LinkedList` exploited inheritance and polymorphism by extending an `AbstractionCollection` class, which provided higher-level functionality and an interface to other collection classes. Lacking this class in C++, we define `LinkedList` as a top-level class. Our C++ version includes a constructor, a destructor (more on this in a moment), the public stack instance methods, a nested node type, and a helper method to allocate and initialize a new node. Here is the code for the class:

```
template <class E> class LinkedList{

    public:

        // Constructor
        LinkedList()
        : top(0), mySize(0) {}

        // Destructor
        ~LinkedList(){
            while (size() > 0)
                pop();
        }

        E peek(){
            return top->data;
        }

        E pop(){
            E element = top->data;
            node * garbage = top;
            top = top->next;
            mySize -= 1;
            delete garbage;    // Must return node storage to the heap
            return element;
        }

        void push(E element){
            top = getNode(element, top);
            mySize += 1;
        }

        int size(){
            return mySize;
        }

    private:

        // The node type for LinkedList
        struct node{
```

(continues)

(continued)

```

        E data;
        node* next;
    };

    // Data members for LinkedStack
    node* top;
    int mySize;

    // Helper method to initialize a new node
    node* getNode(E data, node * next){
        node * newNode = new node;
        newNode->data = data;
        newNode->next = next;
        return newNode;
    }

};

```

The notation `template <class E>` in the class header introduces the type parameter `E` for the stack's element type. This parameter will be filled in with an actual C++ type in the client program's code at compile time. The constructor and public instance methods are similar to those of the `Complex` class discussed earlier.

However, because the `LinkedStack` class utilizes dynamic storage for its nodes, it should also include a destructor. The job of this method is to return each node to the system heap when the `LinkedStack` object is no longer accessible from the client program (for example, when a function that creates a temporary stack returns from its call). The destructor accomplishes this by popping the stack until it is empty. The `pop` method, in turn, manually returns the node just removed from the linked structure to the system heap, by using the `delete` operator. Because C++ has no automatic garbage collection, failure to recycle storage in this manner constitutes a memory leak, which can lead to memory problems in production situations.

Note also that the node type is a C-style `struct`, but could have been defined as a C++ class instead. Because a node is just a private container for a datum and a link, the use of a `struct` and a private method `getNode` that builds and returns a pointer to the initialized node works just as well.

Finally, the use of explicit C-style pointers rather than references illustrates another difference between Java and C++. Even if we had defined a `Node` class in C++, we would still have had to use the `*` operator to indicate a link to the next node and a pointer variable to the node itself.

5.4.3 Static Binding, Dynamic Binding, and Virtual Functions

Dynamic binding of member functions is provided as an option in C++, but it is not the default. All member functions are statically bound, at compile time. Only functions defined using the keyword `virtual` are candidates for dynamic binding. For example, if we want to redefine an `area` function for a square derived from a rectangle, we would do so as follows in C++:

```

class Rectangle{
public:

    virtual double area(){
        return length * width;
    }

    ...

private:

    double length,width;
};

class Square : public Rectangle{

public:

    double area(){          // redefines rectangle::area dynamically
        return width * width;
    }

    ...
};

```

Of course, what we really want in this example is an abstract area function in an abstract class `ClosedFigure`. This can be achieved in C++ by the use of a so-called **pure virtual declaration**:

```

class ClosedFigure{          // an abstract class

public:

    virtual double area() = 0; // pure virtual

    ...
};

class Rectangle : public ClosedFigure{

public:

    double area(){
        return length * width; }
}

```

(continues)

(continued)

```

    ...

    private:

        double length,width;
};

class Circle : public ClosedFigure{

    public:

        double area(){
            return PI*radius*radius; }

    ...

    private

        double radius;
};

```

The 0 in a virtual function declaration indicates that the function is null at that point, that is, has no body. Thus, the function cannot be called, hence is abstract, and also renders the containing class abstract. Pure virtual functions must be declared `virtual`, since they must be dynamically bound during execution and overridden in a derived class.

Note in the preceding code that the overriding definitions of `area` in `Circle` and `Rectangle` do not repeat the keyword `virtual` (although it is permissible to do so). This is because, once a function is declared as `virtual`, it remains so in all derived classes in C++.

Returning to the issue of dynamic binding in C++, it is not sufficient to declare methods as `virtual` to enable dynamic binding; in addition, objects themselves must be either dynamically allocated or otherwise accessed through a reference. To see this in action, consider the following code:

```

(1) class A
(2) {
(3) public:
(4)     void p()
(5)     { cout << "A::p\n" ;
(6)     }

(7)     virtual void q()
(8)     { cout << "A::q\n" ;
(9)     }

```

(continues)

(continued)

```

(10) void f()
(11) { p();
(12)   q();
(13) }
(14) };

(15) class B : public A
(16) {
(17) public:
(18)   void p()
(19)   { cout << "B::p\n" ;
(20)   }
(21)   void q()
(22)   { cout << "B::q\n" ;
(23)   }
(24) };

(25) int main()
(26) { A a;
(27)   B b;
(28)   a.f();
(29)   b.f();
(30)   a = b;
(31)   a.f();
(32) }

```

The somewhat surprising output from this program is:

```

A::p
A::q
A::p
B::q
A::p
A::q

```

In the call `b.f()` (line 29) the inherited function `A.f` is called. Inside this function in `A` are calls to `p` and `q` (lines 11 and 12). Since `p` is not virtual, dynamic binding does not apply, and this call is resolved at compile time to `A::p()`. On the other hand, the call to `q` is virtual, and the implicit object parameter is used to dynamically resolve the call during execution. Thus, the call to `q` is implicitly a call `this->q()`, and in the call `b.f()`, `this` is the address of `b`, so `B::q` appears on the fourth line of the above output. On the other hand, the assignment `a = b` (line 30) simply copies the data of `b` to `a`, and does not change the address of `a` or the `this` pointer in the call to `a.f()` (line 31). Thus, `A::p` and `A::q` are again called.

If, on the other hand, we had changed the main program above to:

```
(33) int main(){
(34)     A* a = new A;
(35)     B* b = new B;
(36)     a->f();
(37)     b->f();
(38)     a = b;
(39)     a->f();
(40)     return 0;
(41) }
```

we would see the output:

```
A::p
A::q
A::p
B::q
A::p
B::q
```

since now the assignment `a = b` (line 38) copies a pointer and changes this pointer in the call to `a->f()` to point to `b`'s object (which is a `B`) instead of the previous object pointed to by `a` (which was an `A`).

C++ offers multiple inheritance using a comma-separated list of base classes, as in

```
class C : public A, private B {...};
```

Multiple inheritance in C++ ordinarily creates *separate* copies of each class on an inheritance path. For example, the declarations:

```
class A {...};
class B : public A {...};
class C : public A {...};
class D : public B, public C {...};
```

provide any object of class `D` with *two* separate copies of objects of class `A`, and so create the inheritance graph shown in Figure 5.10.

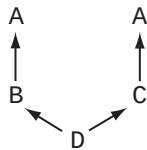


Figure 5.10: Inheritance graph showing multiple inheritance

This is an example of **repeated inheritance**. To get a single copy of an A in class D, one must declare the inheritance using the **virtual** keyword:

```
class A {...};  
class B : virtual public A {...};  
class C : virtual public A {...};  
class D : public B, public C {...};
```

This achieves the inheritance graph shown in Figure 5.11.

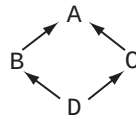


Figure 5.11: Inheritance graph showing shared inheritance

This is an example of **shared inheritance**. Resolution of method conflicts created by multiple (shared) inheritance is explored in Exercise 5.16.

5.5 Design Issues in Object-Oriented Languages

In this section, we survey a few of the issues surrounding the introduction of object-oriented features into a programming language and summarize the different approaches taken in the three languages we have studied.

Object-oriented features represent at their heart dynamic rather than static capabilities (such as the dynamic binding of methods to objects), so one aspect of the design of object-oriented languages is to introduce features in such a way as to reduce the runtime penalty of the extra flexibility. In a dynamically oriented (and usually interpreted) language like Smalltalk, this is less important, but in languages like C++, and to a certain extent in Java, the runtime penalty of their object-oriented features over the non-object-oriented features of their cousins (such as C and Algol) is an important aspect of their overall design. In particular, C++ was designed with efficiency as a primary goal.

One feature promoting efficiency has already been mentioned in the section on C++: Member functions whose implementations are included in the class definition are automatically inline functions in C++. This avoids the penalty of function call during execution.

Other issues that arise in connection with the efficient design of object-oriented languages involve the proper organization of the runtime environment as well as the ability of a translator to discover optimizations. Discussion of such implementation issues is delayed until the next section.

We note also that this chapter discusses design issues only as they relate to overall language design, not program design. Designing object-oriented programs in a way that takes maximum advantage of an object-oriented language involves complex software engineering issues and is outside the scope of this book. Several of the references at the end of the chapter contain more information on object-oriented software design.

5.5.1 Classes vs. Types

The introduction of classes into a language with data types means that classes must be incorporated in some way into the type system. There are several possibilities, including the following three:

1. Classes could be specifically excluded from type checking. Objects would then become typeless entities, whose class membership would be maintained separately from the type system—generally at runtime using a tagging mechanism. This method of adding classes to an existing language is the simplest and has been used in a few languages, such as Objective-C (see the references at the end of the chapter).
2. Classes could become type constructors, thus making class declarations a part of the language type system. This is the solution adopted by C++ and a number of other languages that add object-oriented facilities to an existing language. In C++ a class is just a different form of record, or structure, type. In this case, the assignment compatibility of objects of descendant classes to objects of ancestor classes must be incorporated into the type-checking algorithms. Specifically, if x is an object of a descendant class of the class of object y , then type checking must permit the assignment $y = x$ but must flag the assignment $x = y$ as an error. This complicates the type-checking algorithm.
3. Classes can also simply *become* the type system. That is, all other structured types are excluded from the language. This is the approach of many languages designed from the beginning as object-oriented, such as Smalltalk. This is also almost true of Java, where the class and the array are the only two structured types; however, there are still the basic types such as `boolean`, `int`, `double`, `char`, and their variants, and type checking must also involve these types. Using classes as the basis for the type system means that, under static typing, the validity of all object assignments can be checked prior to execution, even though exact class membership is not known until execution time.¹

5.5.2 Classes vs. Modules

Classes provide a versatile mechanism for organizing code that also encompasses some of the desired features of abstract data type mechanisms: Classes provide a mechanism for specifying interfaces, for localizing code that applies to specific collections of data, and for protecting the implementation through the use of private data. Unfortunately, except in Java, classes do not allow the clean separation of implementation from interface, nor do classes protect the implementation from exposure to client code. Thus, code that uses a class becomes dependent on the particular implementation of the class even if it cannot make use of the details of that implementation.

Another problem with the use of classes alone to structure programs is that classes are only marginally suited for controlling the importing and exporting of names in a fine-grained way. In simple cases, a class can be used as a namespace or scope for a collection of utilities. For example, the Java `Math` class

¹ There are always situations, such as downcasting, where exact class membership must be tested in some way. We mention the Java `instanceof` operation. C++ has a related `dynamic_cast` operation that tests class membership while performing a cast during execution.

consists entirely of static constants and methods and is, therefore, really a namespace masquerading as a class. However, it is not possible to import names from other classes and reexport a selection of them without some effort, so classes do not function well as static namespaces or modules.

For this reason, object-oriented languages often include module mechanisms that are independent of the object-oriented mechanisms. The principle example is C++; it has the namespace mechanism, which is orthogonal to the class mechanism. Java, too, has a package mechanism and interfaces that allow classes to be collected into groups. When defined in a package, classes can be given special names by which their contents can be accessed (typically these names are also associated with a directory structure).

Sometimes object-oriented mechanisms are more closely related to a module structure. For instance, Ada95 introduced a number of object-oriented mechanisms (which we have not studied in this text), and the semantics of declarations using these mechanisms often depend on their relationship to the Ada packages in which they are defined.

Thus, the issue of what kind of module mechanism fits best in an object-oriented setting, and how best to integrate the two notions of class and module, remains, to a certain extent, open.

C++ also has a mechanism that allows a class to open up details of its implementation to another class or function: The other class or function is listed by the class as a `friend`. For example, in a C++ implementation of a complex data type, the problem arises that a binary operation such as addition acquires an unusual syntax when written as a method of class `Complex`, namely,

```
x.operator+(y)
```

(Fortunately, C++ does allow the substitute syntax `x + y`.) An alternative solution, and the only one in some earlier C++ implementations), is to declare the `+` operation outside the class itself as a `friend` (so that it can access the internal data of each `Complex` object) as follows:

```
class Complex
{
private:
    double re,im;

public:
    friend Complex operator+(Complex,Complex);
    ...
};

Complex operator+(Complex x, Complex y)
{ return Complex (x.re+y.re,x.im+y.im); }
```

5.5.3 Inheritance vs. Polymorphism

There are four basic kinds of polymorphism:

1. Parametric polymorphism, in which type parameters may remain unspecified in declarations. ML type variables, Ada generic packages, C++ templates, and Java generic interfaces and classes are examples of parametric polymorphism.
2. Overloading, or ad hoc polymorphism, in which different function or method declarations share the same name and are disambiguated through the use of the types of the parameters in each declaration (and sometimes the types of the returned values). Overloading is available in C++, Java, Haskell, and Ada, but not C or ML.
3. Subtype polymorphism, in which all the operations of one type can be applied to another type.
4. Inheritance, which is a kind of subtype polymorphism. Inheritance can also be viewed as a kind of overloading on the type of the implicit object parameter (the `this` pointer). However, this form of overloading is restricted in that it overloads only on this one parameter. Thus, most object-oriented languages also provide overloading on the explicit parameters of methods.

The problem with the mixture of inheritance and overloading is that it does not account for binary (or n -ary) methods that may need overloading based on class membership in two or more parameters. This is called the **double-dispatch** or **multi-dispatch** problem.

We noted a good example of this problem earlier in this chapter when defining the `Complex` class. Take, for example, complex addition, which can be written in Java as `x.add(y)`, for `x` and `y` `Complex` objects. This can be written even more appropriately as `x.operator+(y)` or `x + y` in C++. Using overloading, this can even be extended to the case where `y` is no longer `Complex` but might be a `double` or an `int`. How might we define `Complex` addition if `x` is a `double` and `y` is a `Complex`? In Java this cannot be done, and in C++ it cannot be done if the `+` operator is defined as a method of the `Complex` class. The way C++ solves this problem is to define a separate, free (overloaded) `+` function taking two `Complex` parameters (see the previous subsection) and then to provide conversions from `int` and `double` to `Complex` (which actually can come simply from the constructors alone).²

This has the decidedly unpleasant effect of splitting up the definition of complex numbers into a class definition and a number of free function definitions.

Several attempts have been made to design an object-oriented language that solves this problem via so-called **multimethods**: methods that can belong to more than one class and whose overloaded dispatch can be based on the class membership of several parameters. One of the more successful attempts is the **Common Lisp Object System (CLOS)**. You can consult the references at the end of this chapter for details.

While overloading and inheritance are related concepts, and, except for the multiple dispatch problem, appear to mesh well in languages such as Java and C++, parametric polymorphism is more

² Constructors taking a single parameter in C++ are called copy constructors. Such constructors provide automatic conversions in ways that we have not studied here; see the C++ references at the end of the chapter.

difficult to integrate, and C++ and Java remain somewhat solitary examples of languages that attempt an integration of the two concepts. As we have noted, Smalltalk and many other object-oriented languages imitate parametric polymorphism using the based-object approach, but this removes the security of static typing from code that uses this form of polymorphism (a similar style of programming is possible in C++ using `void*` pointers, which also circumvents static type checking).

5.6 Implementation Issues in Object-Oriented Languages

In this section, we discuss a few of the issues encountered when implementing an object-oriented language. We also present several methods that allow object-oriented features to be implemented in an efficient way. Since these are primarily issues of efficient code generation by a compiler, they are less applicable to a dynamic, primarily interpreted language such as Smalltalk.

5.6.1 Implementation of Objects and Methods

Typically, objects are implemented exactly as record structures would be in C or Ada, with instance variables representing data fields in the structure. For example, an object of the following class (using Java syntax),

```
class Point{
    ...
    public void moveTo(double dx, double dy){
        x += dx;
        y += dy;
    }
    ...
    private double x,y;
}
```

could be allocated space for its instance variables `x` and `y` just like the structure:

```
struct{
    double x;
    double y;
};
```

by allocating space for `x` and `y` sequentially in memory, as shown in Figure 5.12.

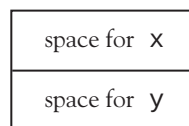


Figure 5.12: Space allocated for a C struct

An object of a subclass can be allocated as an extension of the preceding data object, with new instance variables allocated space at the end of the record. For example, given the declaration (again in Java syntax):

```
class ColoredPoint extends Point{  
    ...  
    private Color color;  
}
```

an object of class ColoredPoint could be allocated as shown in Figure 5.13.

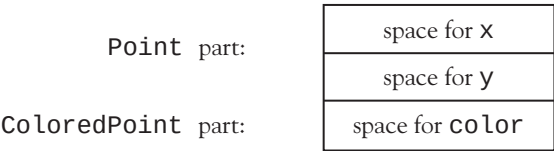


Figure 5.13: Allocation of space for an object of class ColoredPoint

Now the instance variables x and y inherited by any ColoredPoint object from Point can be found at the same offset from the beginning of its allocated space as for any point object. Thus, the location of x and y can be determined statically, even without knowing the exact class of an object; you just need to know that it belongs to a descendant class of Point.

Methods can also be implemented as ordinary functions. Methods do differ from functions in two basic ways. First, a method can directly access the data of the current object of the class. For example, the moveTo method of class Point changes the internal state of a point by assigning to the instance variables x and y. To implement this mechanism, we simply view a method as a function with an implicit extra parameter, which is a pointer that is set to the current instance at each call. For example, the moveTo method of class Point given can be implicitly declared as follows:

```
procedure moveTo(Point* p, double dx, double dy);
```

In this case, each call p.moveTo(dx, dy) would be interpreted by the translator as moveTo(p, dx, dy). Note that, as before, the instance variables of the object pointed to by p can be found by static offset from p, regardless of the actual class membership of the object at the time of execution.

A more significant problem arises with the dynamic binding of methods during execution, since this depends on the class membership of the current object when the call is made. This problem is discussed next.

5.6.2 Inheritance and Dynamic Binding

In the implementation of objects that we have described, only space for instance variables is allocated with each object; allocation of methods is not provided for. This works as long as methods are completely equivalent to functions, and the location of each is known at compile time, as is the case for ordinary functions in an imperative language.

A problem arises with this implementation when dynamic binding is used for methods, since the precise method to use for a call is not known except during execution. A possible solution is to keep all dynamically bound methods³ as extra fields directly in the structures allocated for each object. Runtime information on the class structure can still be maintained efficiently using a statically constructed representation of the class hierarchy, typically some kind of table, with an extra pointer to give access to the appropriate position in the table. If it is possible to create classes dynamically, then this table must be dynamically maintained, but it is still possible to make this reasonably efficient using hash tables.

5.6.3 Allocation and Initialization

Object-oriented languages such as C++ can maintain a runtime environment in the traditional stack/heap fashion of C, as will be described in Chapter 10. Within such an environment, it is possible to allocate objects on either the stack or the heap. However, because objects are generally dynamic in nature, it is more common for them to be allocated on the heap than the stack. Indeed, that is the approach adopted in Java and Smalltalk.

C++, by contrast, permits an object to be allocated either directly on the stack or as a pointer. In the case of stack allocation, the compiler is responsible for the automatic generation of constructor calls on entry into a block and destructor calls on exit, while in the case of heap allocation of pointers, explicit calls to `new` and `delete` cause constructor and destructor calls to be scheduled. This means that C++ can maintain exactly the same runtime environment as C, but with the added burden to the compiler that it must schedule the constructor and destructor calls, as well as create and maintain locations for temporary objects.

Smalltalk and Java, on the other hand, have no explicit deallocation routines, but require the use of a garbage collector to return inaccessible storage to the heap. The execution overhead and added runtime complexity of such a requirement are significant, but techniques such as generational garbage collection can be used to reduce the penalty. See Chapter 10 for a further discussion.

During the execution of initialization code, it is necessary to execute the initialization code of an ancestor class before current initializations are performed. A simple example of this arises when instance variables defined in an ancestor need to be allocated space before they can be used in the initialization of a descendant. In C++ and Java, the calls to initialization code of ancestors are scheduled automatically by a compiler. In C++, where multiple inheritance can create several different paths from a descendant class to an ancestor class, initialization code is performed according to the topological sort of the dependency graph determined by the order in which parent classes are listed in each derived class declaration.

In addition to the question of the order of initializations, especially in a language with multiple inheritance, problems can arise related to the existence of appropriate constructors when they are to be supplied automatically. For example, in Java, when there is no default constructor for a superclass, an explicit constructor call must be supplied in all constructors of subclasses.

³ Recall that *all* methods are (potentially) dynamically bound in Java and Smalltalk, but not in C++.

Exercises

- 5.1 Obtain an implementation of Smalltalk and enter the `Complex` class discussed in Section 5.2, and then add methods to add and subtract complex numbers (you just compute the sums and differences of the real parts and the imaginary parts, respectively).
- 5.2 Smalltalk's `Integer` class includes a `gcd:` method that uses a loop to compute the greatest common divisor of two integers. Add a new method named `recursiveGcd:` to the `Integer` class. This method should compute the greatest common divisor of two integers using a recursive strategy. Note that Smalltalk uses the `//` operator for integer remainder.
- 5.3 Add an `ArrayQueue` class to Smalltalk's `Collection` hierarchy. This class should have the same functionality as the `LinkedList` class developed in Section 5.2. However, the new class should use an `OrderedCollection` object to contain the elements.
- 5.4 Implement a `LinkedListPriorityQueue` class in Smalltalk. The objects added to this type of queue must recognize the `<`, `=`, and `>` operators. The objects are ranked in the queue from smallest (at the front) to largest (at the rear). If two objects are equal, they are inserted in standard FIFO queue order. Your implementation should make maximum use of the `LinkedList` class developed in Section 5.2.
- 5.5 Compare and contrast the types of problems for which functional programming languages and object-oriented programming languages are best suited, using at least two example problems.
- 5.6 The designers of Java distinguish the primitive types (scalars) from the reference types (all other types of values) in the language. Discuss the costs and benefits to the programmer of this decision.
- 5.7 In the design of collection classes, some collections are subclasses of other collections (the “is-a” relation), whereas other collections contain or use other collections in their implementations (the “has-a” relation). Pick an example of each type of design and explain why the design decision is preferable to the alternative.
- 5.8 In Smalltalk, the `Dictionary` class is a subclass of the `Set` class, whereas in Java, the `Map` classes are not included in the `Collection` class hierarchy as subclasses. Compare and contrast the costs and benefits of these two design decisions.
- 5.9 Describe the difference between instance methods and class methods in Smalltalk and give an example of each.
- 5.10 A class variable is visible to and shared by all instances of a class. How would such a variable be used in an application?
- 5.11 What is a constructor in Java? How can constructors be overloaded to support code reuse?
- 5.12 A static method in Java is similar to a class method in Smalltalk (see Section 5.3.4 for an example). Define a static method `getInstance` in the `Complex` class developed in Section 5.3. This method expects two floating-point numbers as arguments and returns a new complex number containing these values. Then, write a code segment that shows how this method might be used.
- 5.13 Explain the difference between the `==` and `=` operators in Smalltalk. How do they illustrate reference semantics?
- 5.14 Describe the difference between abstract classes and concrete classes, giving an example of each.

- 5.15 Explain why the methods `equals` and `toString` are defined in Java's `Object` class.
- 5.16 Explain why a programmer might include the Java methods `equals` and `toString` when defining a new class.
- 5.17 Complete the implementation of the Java `filter` method in the `Iterators` class discussed in Section 5.3.4, and then run the example application (after adding the other resources developed in that section).
- 5.18 Describe how dynamic binding of methods in an object-oriented language works, with a specific example from Smalltalk, Java, or C++.
- 5.19 When is static binding of methods possible in an object-oriented language? Give specific examples from Java and C++.
- 5.20 Compare and contrast how data are encapsulated and information is hidden in Smalltalk, Java, and C++.
- 5.21 Explain the difference between a class and an interface in Java, using at least one example.
- 5.22 Study the `Comparable` interface in Sun's Java documentation, and modify the `Complex` class developed in Section 5.3 so that it implements this interface. Explain why this move will enable a programmer to sort arrays or lists of `Complex` numbers using methods in Java's `Arrays` or `Collections` classes.
- 5.23 Java and C++ support generic collections with type parameters, whereas Smalltalk does not. Does that fact place the Smalltalk programmer at a disadvantage? Why or why not?
- 5.24 The class diagrams of Figures 5.6 and 5.7 show Java's `List` interface and its implementing classes. In Section 5.3, we developed a `LinkedStack` class as a subclass of `AbstractCollection`. An alternative implementation, called `ArrayStack`, includes the same methods as `LinkedStack`. Draw a class diagram that places these two classes and a new interface called `TrueStack` in the appropriate places in the hierarchy. (*Hints:* The `TrueStack` interface should extend the `Collection` interface. The stack classes should extend the `AbstractCollection` class and also implement the `TrueStack` interface.)
- 5.25 Modify the `LinkedStack` class and implement the `ArrayStack` class and the `TrueStack` interface mentioned in the previous exercise.
- 5.26 List the types of iterators supported in Smalltalk and explain how they work.
- 5.27 The enhanced `for` loop in Java depends on an iterator. Explain how this works.
- 5.28 Maps and filters exist in functional languages and in Smalltalk. Explain how these software constructs work in both contexts.
- 5.29 If `x` is an object of class `A`, and `y` is an object of a class `B` that is a subclass of `A`, after the assignment `x = y`, why is `x.m()` illegal when `m` is a method of `B` but not `A`?
- 5.30 Describe the two meanings of the keyword **virtual** in C++ discussed in the text.
- 5.31 What is the difference between a type and a class?
- 5.32 Some have claimed that deep inheritance hierarchies make system maintenance more difficult, not easier. Explain how that could be true.
- 5.33 A `print` operation is a good candidate for a dynamically bound method. Why?

5.34 Given the following code in Java:

```
class A{

    public void p(){
        System.out.println("A.p");
    }

    public void q(){
        System.out.println("A.q");
    }

    public void r(){
        p();
        q();
    }
}

class B extends A{

    public void p(){
        System.out.println("B.p");
    }
}

class C extends B{

    public void q(){
        System.out.println("C.q");
    }

    public void r(){
        q();
        p();
    }
}

...
A a = new B();
a.r();
a = new C();
a.r();
```

What does this code print? Why?

5.35 Given the following code in C++:

```
class A{
    public:
        virtual void p(){
            cout << "A.p" << endl;
        }
        void q(){
            cout << "A.q" << endl ;
        }
        virtual void r(){
            p();
            q();
        }
};

class B : public A{
    public:
        void p(){
            cout << "B.p" << endl;
        }
};

class C : public B{
    public:
        void q(){
            cout << "C.q" << endl;
        }
        void r(){
            q();
            p();
        }
};
```

(continues)

(continued)

```

...
A a;
C c;
a = c;
a.r();
A* ap = new B;
ap->r();
ap = new C;
ap->r();

```

What does this code print? Why?

- 5.36 Explain in detail why multiple interface inheritance in Java does not suffer from the same problems as multiple class inheritance in C++.
- 5.37 Meyer [1997] distinguishes overloading from parametric polymorphism as follows (Ibid., p. 38): “Overloading is a facility for client programmers: it makes it possible to write the same client code when using different implementations of a data structure provided by different modules. Genericity [parameterization] is for module implementors: it allows them to write the same module code to describe all instances of the same implementation of a data structure, applied to various types of objects.” Discuss this view of polymorphism.

Notes and References

A good general reference for object-oriented programming techniques is Meyer [1997], which uses the interesting object-oriented language Eiffel, which you have not studied in this chapter. Budd [1997] and Booch [1994] describe object-oriented principles in a language-independent way. Bruce [2002] provides a principled examination of types and classes. Java, of course, has many references, but two that emphasize object-oriented principles are Arnold et. al [2000] and Horstmann [2006]. C++ is described in Stroustrup [1997] and Lippman and Lajoie [1998]. Books on Smalltalk proper include Lalonde [1994], Lewis [1995], Sharp [1997], and Lambert and Osborne [1997].

Other interesting object-oriented languages that are not studied here are described in the following references. The Dylan language is described in Feinberg et al. [1996]. Eiffel is a carefully designed language with a Pascal-like syntax (Meyer [1997]). Modula-3 is a Modula-2 derivative with object-oriented extensions (Cardelli et al. [1989], Nelson [1991], and Cardelli et al. [1992]). Objective C is an extension of C with object-oriented features designed by Cox [1984, 1986]. An object-oriented extension of Pascal is described in Tesler [1985]. Object-oriented versions of LISP include CLOS (Gabriel, White, and Bobrow [1991]; Bobrow et al. [1988]); Loops (Bobrow and Stetik [1983]); and Flavors (Moon [1986]).

Information on the use of C++ templates and their interaction with object-oriented techniques can be found in Alexandrescu [2001], Koenig and Moo [1996], Josuttis [1999], and Stroustrup [1997].

Many research papers, as well as reports on object-oriented programming techniques, have appeared in OOPSLA [1986ff]. An interesting application of object-oriented techniques to the sieve of Eratosthenes in C++ is given in Sethi [1996].